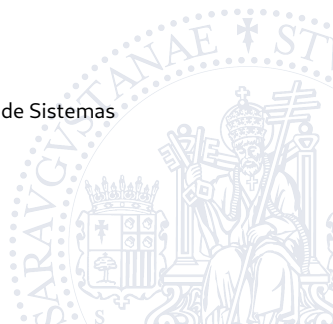


1. El proceso de pruebas

Verificación y validación (cód. 30244)

Miguel Ángel Latre
Javier Nogueras Iso

Departamento de Informática e Ingeniería de Sistemas
Universidad de Zaragoza



Índice

1. Conceptos básicos
2. Fundamentos de TMAP
3. Gestión global del proceso de pruebas
4. Pruebas de sistema y aceptación
5. Pruebas de desarrollo

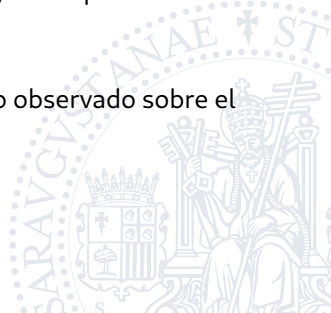


1. Conceptos básicos
2. Fundamentos de TMAP
3. Gestión global del proceso de pruebas
4. Pruebas de sistema y aceptación
5. Pruebas de desarrollo



1. Conceptos básicos

- Defecto (*fault, bug*)
 - La causa mecánica o algorítmica de un error
- Error
 - El sistema está en un estado tal que el siguiente procesamiento del sistema llevará a un fallo
- Fallo (*failure*)
 - Cualquier desviación del comportamiento observado sobre el especificado

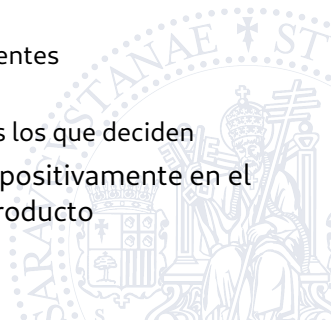


El proceso de pruebas

- Es un proceso mediante el cual se **aplican** una serie de **métodos** (ocasionalmente utilizando herramientas) que permiten obtener un conjunto de medidas para **verificar y validar el funcionamiento** requerido del software
- Permite describir y aconsejar sobre la calidad y los riesgos relacionados de un producto software
 - La calidad es la totalidad de características que un producto o servicio ofrece para satisfacer las necesidades establecidas o implícitas
 - Una característica de la calidad es una propiedad de un sistema de información:
 - funcionalidad (corrección y completitud), efectividad, eficiencia, rendimiento, continuidad, portabilidad, ...
 - Las pruebas es un proceso a incluir dentro del sistema de gestión de la calidad de una organización (es una de las medidas que permiten conseguir un software de calidad)

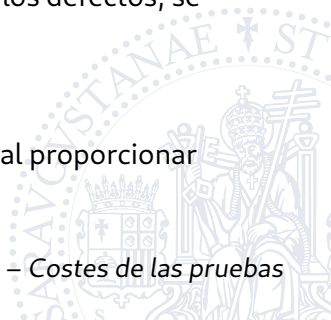
Reflexiones sobre el proceso de pruebas

- Son algo distinto a la implementación aunque haya relación
- No son una fase de pos-desarrollo
 - Por su importancia tienen su propio ciclo de vida que se inicializa en la captura de requisitos y evoluciona en paralelo al desarrollo
- No son un entrenamiento para la utilización/administración del software desarrollado
 - Aunque se presten a ello son tareas diferentes
- No deciden si se debe aceptar algo
 - Solo aconsejan sobre la calidad, son otros los que deciden
- Las pruebas no son baratas, pero influyen positivamente en el proceso de desarrollo y en la calidad del producto



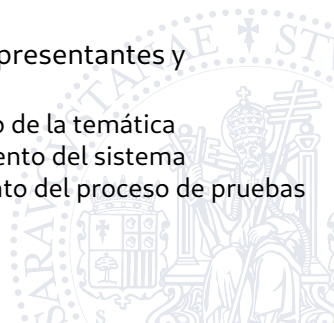
Beneficios de las pruebas

- Al encontrar defectos previamente a la entrega de un sistema, se evitan los costes de rehacer un sistema (y los consecuentes daños) cuyos defectos se detectasen en producción
 - Ejemplos de daños consecuentes:
 - Pérdida de ingresos, pérdida de reputación, reclamación de compensaciones
- Y aunque no se resuelvan completamente los defectos, se minimizan los daños de producción
 - *Declare the bug to be a feature*
- Se tiene/gana confianza en el producto
- Se facilita una buena gestión del proyecto al proporcionar información sobre el progreso y calidad
- Desde un punto de vista económico
 - *Beneficios de las pruebas = Daño evitado – Costes de las pruebas*



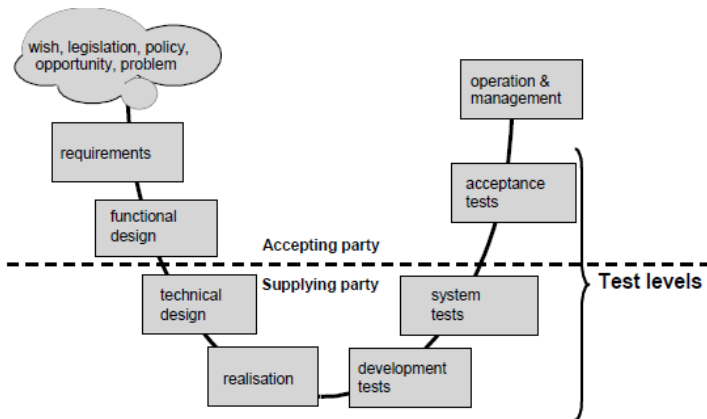
¿Quién ejecuta las pruebas?

- Cualquiera de las partes involucradas en un sistema de información
 - Representantes del cliente: directivos, usuarios, administrador de sistemas, ...
 - Representantes del equipo/empresa de desarrollo
 - Profesionales de las pruebas
- Se recomienda que haya una mezcla de representantes y profesionales de las pruebas
 - Representantes del cliente: conocimiento de la temática
 - Representantes del desarrollo: conocimiento del sistema
 - Profesionales de las pruebas: conocimiento del proceso de pruebas



Niveles de pruebas y responsabilidades

- En el contexto de las pruebas, se hace una distinción de responsabilidades
 - Parte proveedora (equipo/empresa de desarrollo)
 - Se asegura de que el producto entregado es el que se debería entregar
 - Parte aceptante (cliente, usuario, administrador, ...)
 - Se asegura de que el producto recibido es el que se quería/necesitaba
- La distinción de responsabilidades se traduce en la agrupación de actividades de pruebas en niveles de pruebas que se gestionan y ejecutan colectivamente
 - Pruebas de desarrollo (parte proveedora)
 - Cumplimiento de especificaciones técnicas, ...
 - Pruebas de sistema (parte proveedora)
 - Cumplimiento de requisitos funcionales, no funcionales, diseño técnico
 - Pruebas de aceptación (parte aceptante)
 - El producto cumple con las expectativas, ...



Tipos de pruebas

- Dentro de un nivel de pruebas se pueden establecer tipos de pruebas
- Un tipo de pruebas agrupa actividades de prueba orientadas a evaluar un sistema de información respecto a un número relacionado de características de la calidad
- Ejemplos de tipos de pruebas
 - Pruebas funcionales (las habituales)
 - Pruebas de regresión
 - Pruebas de usabilidad
 - Pruebas de rendimiento
 - Pruebas de portabilidad
 - ...



¿Cómo gestionamos las pruebas?

- De forma *ad hoc*
 - Se han programado algunas actividades de prueba pero no hay un plan sólido, no se priorizan los riesgos, ...
- O utilizando una metodología que nos guíe en las tareas a realizar
 - Se proporciona una estructura que nos dice claramente *qué, quien, cuando* y en *qué secuencia* se debe realizar
 - Se cubre el sistema de información de forma completa y abordando todos los aspectos relevantes
 - Se gestionan las actividades teniendo en cuenta el tiempo, el coste económico y la calidad

¿Por qué una metodología/estándar de pruebas del software?

- Cuerpo de conocimiento como base para esta disciplina profesional
 - Alto grado de acuerdo internacional
 - Adaptable
- Beneficios para el cliente:
 - Confianza en el proveedor
 - "Buena" práctica industrial
 - Vinculación contractual
- Beneficios para el proveedor:
 - Comunicación – terminología común
 - Certificación/acreditación, marketing
 - Cualificación profesional, formación
 - Mejora continua, interoperabilidad, consistencia...



Algunas propuestas metodológicas

- Agencia Espacial Europea (ESA) – Guías de Ing. Software a nivel de procesos

ACTIVITY PLAN	USER REQUIREMENTS REVIEW		SOFTWARE REQUIREMENTS DEFINITION		ARCHITECTURAL DESIGN		DETAILED DESIGN AND PRODUCTION	
	Activity	Output	Activity	Output	Activity	Output	Activity	Output
SOFTWARE PROJECT MANAGEMENT	Estimate project cost Plan SR phase WBS & staffing Outline plan for whole project	SPMP/SR	Estimate project cost to 30% accuracy Plan AD phase WBS & staffing	SPMP/AD	Estimate project cost to 10% accuracy Plan DD phase WBS & Staffing	SPMP/DD	Detail DD phase WBS Plan TR phase WBS & Staffing	SPMP/DD updates SPMP/TR
SOFTWARE CONFIGURATION MANAGEMENT	Define SR phase procedures for: - documents - CASE tool products - prototype code	SCMP/SR	Define AD phase procedures for: - documents - CASE tool products - prototype code	SCMP/AD	Define DD phase procedures for: - documents - CASE tool products - deliverable code	SCMP/DD	Define operational environment procedures for: - documents - deliverable code	SCMP/TR
SOFTWARE VERIFICATION AND VALIDATION	Define SR phase review and traceability procedures Plan acceptance tests	SVVP/SR SVVP/IAT	Define AD phase review and traceability procedures Plan system tests	SVVP/AD SVVP/ST	Define DD phase review and traceability procedures Plan integration tests	SVVP/DD SVVP/IT	Define acceptance tests Define system tests Define integration tests Plan and define unit tests	SVVP/IAT updates SVVP/ST updates SVVP/IT updates SVVP/IUT
SOFTWARE QUALITY ASSURANCE	Plan SR phase monitoring activities Outline plan for whole project	SQAP/SR	Plan AD phase monitoring activities	SQAP/AD	Plan DD phase monitoring activities	SQAP/DD	Plan TR phase monitoring activities	SQAP/TR

Figure 1.1: Software Management Plans

● Agencia Espacial Europea (ESA) – Guías de Ing. Software a nivel de producto

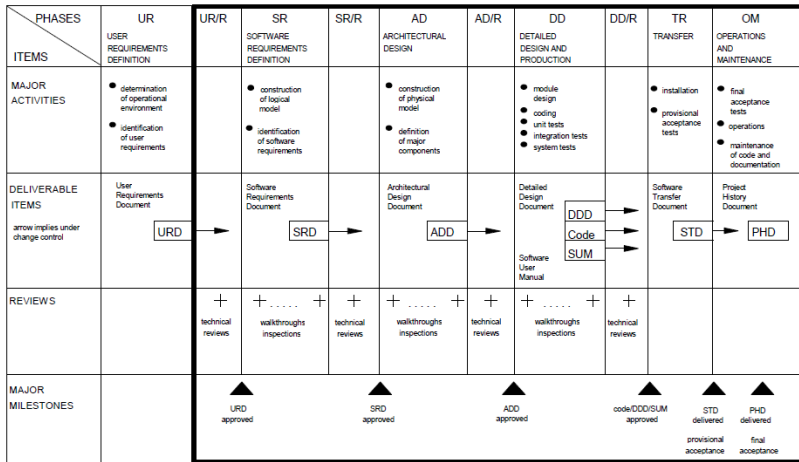
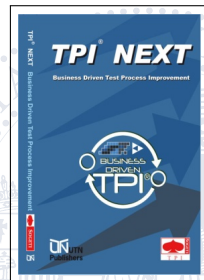
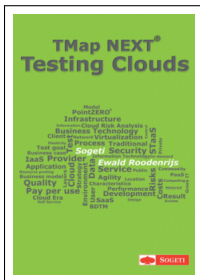
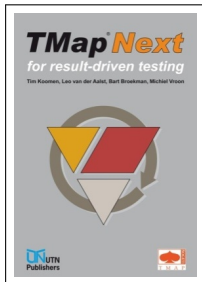


Figure 1.2: The Software Life Cycle Model

https://www.esa.int/TEC/Software_engineering_and_standardisation/

- TMAP Next = Test Management Approach

- metodología para la gestión pruebas que vamos a seguir en este curso
- propuesta por la empresa holandesa Sogetti (desde 1995)
- material asociado a la metodología en <https://www.tmap.net/>
- metodologías relacionadas
 - TPI Next: orientada a la mejora de los procesos de prueba
 - TMAP Next – Testing clouds: metodología de pruebas para computación en la nube



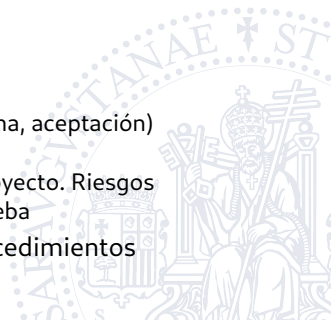
● Test Maturity Model Integration (TMMi)

- <https://www.tmmi.org/>
- Modelo de procesos de pruebas con 5 niveles de madurez pensado para organizaciones que prestan gran importancia a las pruebas
 - Orientado a la mejora de procesos de pruebas
 - Basado en la experiencia de otras propuestas: TMAP, TPI, CMMI

1- Initial	2 - Managed	3 - Defined	4 – Measured	5 - Optimization
(procesos de pruebas caóticos, o no evaluados, etc.)	(gestión básica de los procesos de pruebas) 2.1 Test Policy and Strategy 2.2 Test Planning 2.3 Test Monitoring and Control 2.4 Test Design and Execution 2.5 Test Environment	(pruebas controladas en el ciclo de vida, con hitos, etc.) 3.1 Test Organization 3.2 Test Training Program 3.3 Test Lifecycle and Integration 3.4 Non-Functional Testing 3.5 Peer Reviews	(medición de procesos) 4.1 Test Measurement 4.2 Product Quality Evaluation 4.3 Advanced Peer Reviews	(mejora continua y control estadístico de procesos) 5.1 Defect Prevention 5.2 Quality Control 5.3 Test Process Optimization

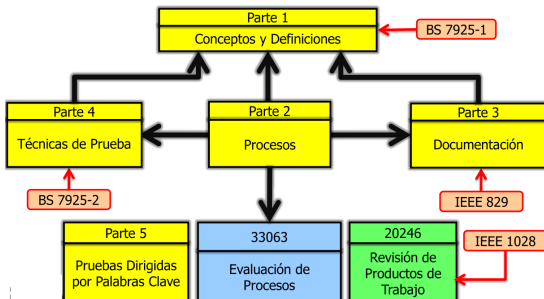
Estándares específicos de organizaciones de normalización

- Estándares BSI (Reino Unido)
 - BS 7925-1, SW Testing : Part 1-Vocabulary
 - BS 7925-2, SW Testing : Part 2-Software Component Testing
- Estándares IEEE testing :
 - IEEE Std 829, Software Test Documentation
 - IEEE Std 1008, Software Unit Testing
- Problemas en estas propuestas
 - Aspectos clave sin cubrir:
 - Pruebas no unitarias (integración, sistema, aceptación)
 - Modelo de procesos explícito
 - Aspectos organizativos y gestión del proyecto. Riesgos
 - Visión más completa de técnicas de prueba
 - Definiciones en conflicto, procesos y procedimientos
 - ¿Qué estándar seguir?



- Solución a los problemas anteriores: ISO/IEC/IEEE 29119
Software Testing

- Alcance : *"This project will produce a software testing standard applicable to all types of software products and software-intensive systems"*
- Propósito : *"to unify and integrate the currently fragmented corpus of normative literature regarding testing that is currently offered by three distinct standards-makers: BSI, IEEE, and ISO/IEC JTC 1/SC 7. The result of the project will be a consistent, unified treatment adopted by all three organizations"*
- Estructura



1. Conceptos básicos
2. Fundamentos de TMAP
3. Gestión global del proceso de pruebas
4. Pruebas de sistema y aceptación
5. Pruebas de desarrollo

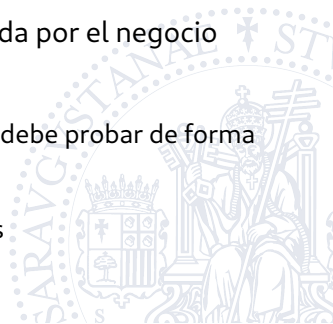


2. Fundamentos de TMAP

- Proceso dirigido por el negocio
 - La economía marca el esfuerzo a realizar, y cuales son los riesgos prioritarios
- Un proceso de pruebas estructurado
 - Nos guía a la hora de responder a las cuestiones típicas de qué/cuando, cómo, con qué y quien (ciclo de vida de pruebas)
- Método completo y adaptable
 - Flexibilidad para adaptar la metodología a distintas situaciones de desarrollo: nuevos desarrollos, mantenimiento, desarrollo propio o basado en software comercial, *outsourcing* de las pruebas, ...

Gestión de pruebas dirigida por el negocio

- La estrategia de pruebas está relacionada con las características y partes del sistema a testear que representan un riesgo si no funcionan adecuadamente en producción
- La metodología de pruebas debe facilitar al cliente controlar el proceso de pruebas y determinar (ayudar en) la selección de estrategias
- Pasos a dar en la gestión de pruebas dirigida por el negocio
 - 1 Identificar los objetivos de las pruebas
 - 2 Determinar los riesgos
 - 3 Determinar si una característica/parte se debe probar de forma detallada o ligera
 - 4 Estimar y planificar
 - 5 Elegir las técnicas de prueba y ejecutarlas
 - 6 Informar sobre progreso, calidad

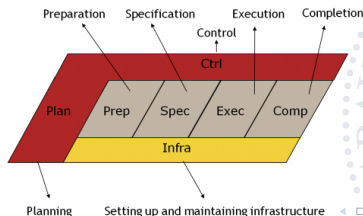


Proceso de pruebas estructurado

- Se compone de:
 - La gestión global del proceso de pruebas
 - El gestor de pruebas, consultando con el cliente y otras partes implicadas, establece una distribución general de lo que debería ser probado en los distintos niveles de pruebas y con qué nivel de detalle
 - Procesos por cada nivel de pruebas
 - Pruebas de aceptación
 - Pruebas de sistema
 - Pruebas de desarrollo

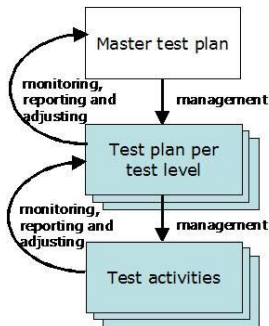


- En la gestión global del proceso de pruebas se distinguen dos fases:
 - Planificación
 - Control
- Para el resto de niveles TMAP propone un ciclo de vida organizado en 7 fases:
 - 1 Planificación (análisis de riesgos, estimar, ...)
 - 2 Control (monitorización y reajuste)
 - 3 Instalación y mantenimiento de la infraestructura
 - 4 Preparación (acordar resultados esperados)
 - 5 Especificación de casos de prueba
 - 6 Ejecución
 - 7 Finalización (aprender, garantizar reutilización de las pruebas en el futuro)



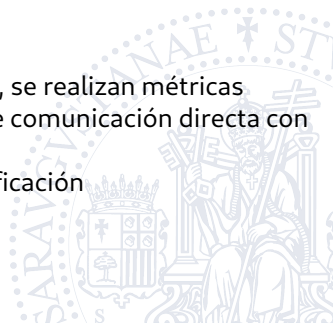
Relación entre los distintos niveles de pruebas

- El plan de pruebas maestro de la gestión global del proceso de pruebas es la base para los planes separados de los niveles de prueba



Método completo y adaptable

- Los elementos de TMAP se utilizan en combinación con los modelos de ciclo de vida de desarrollo
- Se puede adoptar la metodología de forma completa o reaprovechar técnicas, plantillas, listas de comprobación, infraestructura, estrategias, fases
- Es reactivo a cambios
 - Se monitoriza el progreso, los resultados, se realizan métricas sobre el proceso de pruebas, se establece comunicación directa con el cliente
 - Si hay algún problema, se rehace la planificación



1. Conceptos básicos
2. Fundamentos de TMAP
3. Gestión global del proceso de pruebas
 - 3.1 Planificación
 - 3.2 Control
4. Pruebas de sistema y aceptación
5. Pruebas de desarrollo



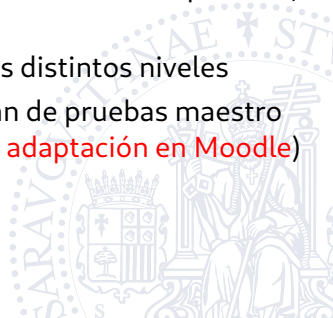
3. Gestión global del proceso de pruebas

- Organizada en dos fases
 - ① Planificación
 - ② Control
- Principales técnicas relacionadas con estas fases
 - Análisis de riesgos de productos
 - Estimación del esfuerzo de las pruebas
 - Elaboración de métricas



3.1. Planificación

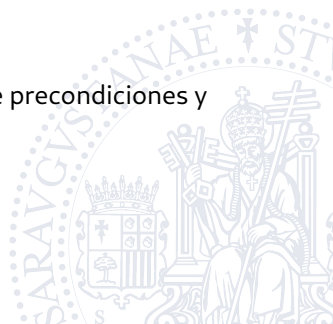
- El autor del plan maestro, habitualmente el gestor de las pruebas, y contando con el cliente, formula la misión (objetivos de las pruebas) teniendo en cuenta resultados, riesgos, tiempo y coste
- Además de la misión, se establece la organización de las pruebas, la infraestructura y la gestión
- El objetivo es no re-inventar la rueda en los distintos niveles
- Se facilita una plantilla para elaborar el plan de pruebas maestro
(ver *Template master test plan* en TMAP y adaptación en Moodle)



3.1.1. Formular la misión

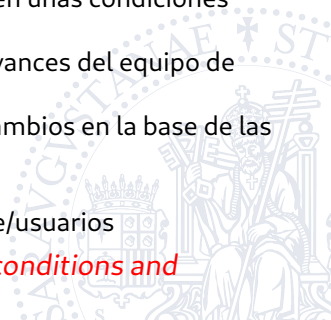
- Propósito
 - Clarificar a todas las partes implicadas el propósito, tareas, responsabilidades y autorizaciones de las pruebas
- Método
 - Clarificar los siguientes aspectos
 - Cliente: la parte que encarga la creación del plan maestro y la ejecución de las pruebas; es importante identificar al gestor del proyecto en representación de los usuarios
 - Proveedor: la parte responsable de crear el plan maestro y/o ejecutar la misión; al coordinador general se le denomina gestor de las pruebas
 - Misión: el gestor de las pruebas ayuda al cliente a formular el propósito de las pruebas de forma clara (ej: probar todo el software –propio o empaquetado– distribuido en el proyecto XX; informar sobre los riesgos de lanzar la versión 1.5 del producto XX a producción)
 - Alcance: describir los límites de las pruebas (sistemas, conversiones, interfaces con otros subsistemas); describir qué está fuera del alcance de las pruebas

- Método ...
 - Clarificar ...
 - Precondiciones: condiciones impuestas sobre el proceso de pruebas por terceras partes (cliente, usuario, proyecto)
 - Asunciones: condiciones externas fuera del control del proceso de pruebas que deben ocurrir para que el proceso de pruebas sea un éxito
- Productos
 - Sección del plan maestro
- Técnicas
 - Lista de comprobación de la existencia de precondiciones y asunciones habituales
- Herramientas (no aplicable)



Lista de comprobación de la existencia de precondiciones y asunciones habituales

- Precondiciones
 - Fecha de entrega o hitos pre-establecidos
 - Restricciones a la utilización de recursos (personal, presupuesto)
 - Utilización obligada de normas y estándares
- Asunciones
 - Disponibilidad del objeto de las pruebas en unas condiciones específicas
 - Se tiene visibilidad de la planificación y avances del equipo de desarrollo
 - Se informa al equipo de pruebas de los cambios en la base de las pruebas
 - Se recibe apoyo del equipo de desarrollo
 - Se recibe apoyo de la organización cliente/usuarios
- Ver ejemplo más detallado en TMAP (*Preconditions and assumptions*)



3.1.2. Entender la misión

- Propósito
 - Investigar el objetivo, organización y proceso del sistema a desarrollar, así como las condiciones que debe cumplir, para controlar mejor otros pasos de la planificación
- Método
 - Determinar la parte aceptante con sus criterios de aceptación
 - Investigar las expectativas: ¿Cuáles son los elementos más críticos: tiempo, dinero, calidad?
 - Estudiar la documentación disponible
 - Realizar entrevistas



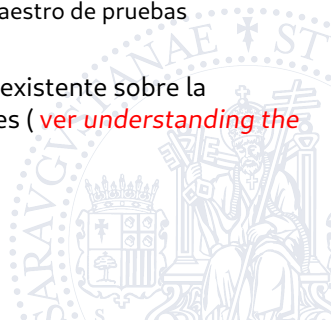
● Productos

- El plan maestro contendrá
 - Aceptantes y criterios de aceptación
 - Referencia a la base de las pruebas (*test basis* = información que define el comportamiento requerido del sistema): en este contexto podemos hacer referencia al documento de requisitos
 - Estándares a seguir: relativos a pruebas, documentos, desarrollo, calidad
 - Documentos relacionados con el plan maestro de pruebas

● Técnicas

- Lista de comprobación de la información existente sobre la organización, el proyecto y las aplicaciones (*ver understanding the assignment en TMAP*)

● Herramientas (no aplicable)



3.1.3. Analizar los riesgos del producto

- Propósito
 - Las partes implicadas y el gestor de las pruebas establecen una visión conjunta de las partes y características más y menos arriesgadas del sistema
- Método
 - 1 Determinar los participantes
 - 2 Determinar la aproximación al análisis de riesgos
 - 3 Preparar sesiones/entrevistas
 - 4 Recolectar y analizar los riesgos del producto
 - 5 Verificar completitud
- Productos
 - Sección y anexo en plan maestro
- Técnicas
 - Definición de características de calidad (**ver lista en Moodle**)
 - Recolectación y análisis de riesgos del producto (integrada dentro del método)
- Herramientas (no aplicable)

Propósito

- El análisis del riesgo de productos consiste en analizar el producto a probar con el propósito de conseguir una visión compartida (entre el gestor de las pruebas y otros actores involucrados) de aquellas características o partes del sistema que tengan mayor o menor riesgo y que el nivel de detalle de las pruebas de estas características y partes sea acorde al riesgo
- El riesgo de un producto se define como la probabilidad de que un producto falle en relación al daño esperado si esto ocurre

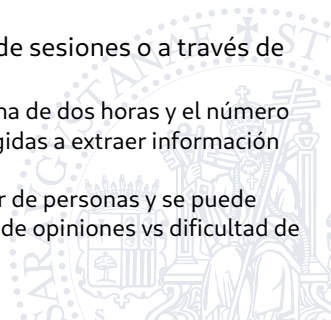
Riesgo del producto = probabilidad de fallo x daño

Probabilidad de fallo = probabilidad de los defectos x frecuencia de uso

- Motivos que aumentan la probabilidad de defectos: desarrollo de funciones complejas o completamente nuevas, inexperiencia de desarrolladores, grandes equipos de desarrollo, ...
- Los daños pueden ser directos (pérdida de ingresos, costes de corrección, daños a terceros) o indirectos (pérdida de imagen, demandas judiciales, ...)

5 pasos para realizar el análisis de riesgos

- 1. Determinar los participantes
 - El gestor de pruebas realiza el análisis en colaboración con la parte aceptante (cliente, administrador, usuario final) y la parte proveedora (arquitectos, desarrolladores, gestor del proyecto, ...)
- 2. Determinar la aproximación al análisis de riesgos. Hay que decidir dos cuestiones principales:
 - ¿Cómo se organizará el análisis: a través de sesiones o a través de entrevistas?
 - Una entrevista tiene una duración máxima de dos horas y el número de entrevistados es limitado (2 o 3); dirigidas a extraer información específica del entrevistado
 - En una sesión participan un grupo mayor de personas y se puede dedicar media jornada; mayor contraste de opiniones vs dificultad de moderar/organizar



- 2. ...

- ¿Qué clasificación de riesgos se va a utilizar?

- Clasificación absoluta: cada riesgo se evalúa de forma separada (A: riesgo alto, B: riesgo medio, C: riesgo bajo)

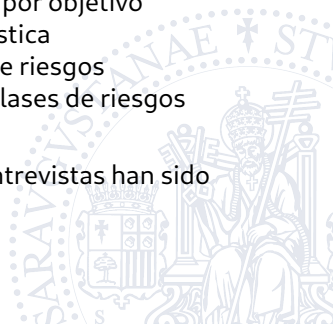
Cada organización puede adaptar esta clasificación

Se pueden adoptar criterios económicos para el daño:
Alto: > 250.000 €
Medio: de 50.000€ a 250.000€
Bajo: < 50.000€

Analizar daño y probabilidad por riesgo		Probabilidad de fallo		
		Alto	Medio	Bajo
Daño en caso de fallo	Alto	A	B	B
	Medio	B	B	C
	Bajo	C	C	C

- Clasificación relativa: cada riesgo se evalúa en relación al resto de riesgos (se ordenan por importancia). Si hay muchos riesgos (ej.: 20), es preferible establecer grupos de riesgos (por ejemplo 3: alto, medio, bajo)

- 3. Preparar sesiones/entrevistas
 - Durante las sesiones y entrevistas puede ayudar contar con listas de daños típicos y situaciones que incrementan las probabilidades de fallo asociadas a características de la calidad (*ver Checklist risk factors per quality characteristic en TMAP*)
- 4. Recolectar y analizar los riesgos del producto. Subactividades:
 - ① Determinar los objetivos de las pruebas
 - ② Determinar las características relevantes por objetivo
 - ③ Distinguir los componentes por característica
 - ④ Por cada característica, rellenar la tabla de riesgos
 - ⑤ Agregar características, componentes y clases de riesgos
- 5. Verificar completitud
 - verificar si los resultados de sesiones y entrevistas han sido suficientemente completos



Determinar los objetivos de las pruebas y las características relevantes por objetivo

- Objetivo de pruebas: objetivo **relevante para el cliente**, a menudo formulado en términos de requisitos de usuario, requisitos funcionales, casos de uso, requisitos no funcionales, factores críticos de éxito, criterios de aceptación, riesgos del producto o directamente una característica de la calidad
- Ejemplo de determinación de objetivos y características

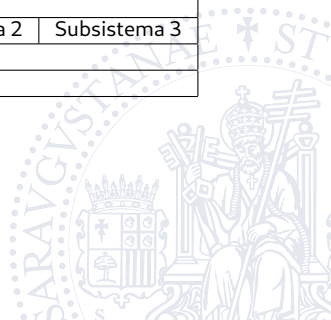
Tipo de objetivo de pruebas	Descripción	Características de calidad
Requisito de usuario / requisito funcional	Se verifica de forma automática la capacidad de pago del solicitante	funcionalidad
Caso de uso	Introducir nuevo cliente	funcionalidad, facilidad de uso
Caso de uso	Abrir cuenta	funcionalidad, facilidad de uso

Tipo de objetivo de pruebas	Descripción	Características de calidad
Requisito no funcional	El sistema debe prever hasta un millón de clientes operando con fluidez	eficiencia
Requisito no funcional / factor crítico de éxito	Mostar una oferta de hipoteca en línea en la pantalla intermedia en menos de 1 minuto	rendimiento
Factor crítico de éxito	El procesamiento de facturación y pedidos funciona correctamente en cualquier plataforma	portabilidad
Criterio de aceptación	Los subsistemas de Gestión de Clientes y Pedidos funcionan correctamente	funcionalidad
Criterio de aceptación	Las pantallas de añadir y editar cliente funcionan correctamente	funcionalidad, facilidad de uso
Criterios de aceptación	La interfaz entre el Sistema de Clientes y el Sistema de Pedidos existente funciona correctamente	conectividad
Riesgo de producto	El cliente recibe facturas incorrectas	funcionalidad
Riesgo de producto	Los usuarios no pueden trabajar con el sistema	facilidad de uso, idoneidad de la infraestructura
Característica de calidad	Facilidad de uso	

Distinguir los componentes por característica

- Ejemplo de determinación de los componentes del sistema desde la perspectiva de la característica a probar

Característica	Componentes		
Funcionalidad	Subsistema 1	Subsistema 2	Subsistema 3
Rendimiento	Batch (por lotes)	En línea	
Facilidad de uso	Sistema global		



Por cada característica, rellenar la tabla de riesgos

- Ejemplo

no es relevante: proceso 1 no implementado en sub. 2

Se requiere prueba de integración del sistema global

Característica: funcionalidad	Componentes	Subsistema 1	Subsistema 2	Sistema global
Objetivos de pruebas	Probabilidad de fallo Daño	Alto	Medio	Bajo
Proceso de negocio 1	Alto	A	-	B
Proceso de negocio 2	Alto	-	B	B
Proceso de negocio 3	Bajo	C	C	C
Clase de riesgo por componente		B	B	B

Clase de riesgo media (B) si A y C tienen misma frecuencia

Clase de riesgo más alta ante dos clases consecutivas con misma frecuencia

Clase de riesgo que ocurre con más frecuencia

Agregar características, componentes y clases de riesgos

● Ejemplo

Característica	Clase de riesgo	Justificación
Funcionalidad		
- subsistema 1	B	probabilidad alta de fallo, utilizado en proceso 1 de gran relevancia y 3 de relevancia baja
- subsistema 2	B	probabilidad media de fallo, utilizado en proceso 2 de gran relevancia y 3 de relevancia baja
- sistema global	B	si los subsistemas 1 y 2 funcionan correctamente, el riesgo de integración es bajo
Facilidad de uso	B	...
Rendimiento		
- en línea	B	
- batch	C	
Seguridad	A	

Se puede eliminar el detalle por componente en el plan maestro si no se tiene claro -> agruparíamos por característica

- En el plan de pruebas maestro existirá una sección con los resultados del análisis de riesgos

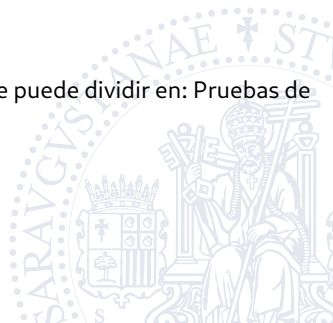
Tipo de objetivo de pruebas	Descripción	Característica
Caso de uso / requisito de usuario	<< Descripción del objetivo de pruebas >>	< funcionalidad, rendimiento, facilidad de uso, idoneidad, etc. >

Característica	Componente	RC	Justificación
Funcionalidad	...	A/B/C	...
Facilidad de uso			
Rendimiento			
Seguridad			
Idoneidad			

RC = clase de riesgo (risk class)
Clase A : riesgo más alto
Clase C : riesgo más bajo

3.1.4. Determinar la estrategia de las pruebas

- Propósito
 - Utilizando como entrada el resultado del análisis de riesgos del producto, se decide que característica/parte del sistema debe ser probado y con qué nivel de detalle en cada nivel de pruebas
- Método
 - Determinar los niveles de pruebas
 - Pruebas de desarrollo. Si es necesario, se puede dividir en: Pruebas de unidad; Pruebas de integración
 - Pruebas de sistema



• ...

• ...

- Pruebas de aceptación (generales)
- Pruebas de aceptación de usuarios: llevadas a cabo por los futuros usuarios en un entorno de producción simulado con el objetivo de verificar **requisitos de usuario**
- Pruebas de aceptación funcional: llevadas a cabo por los futuros usuarios en un entorno de producción simulado con el objetivo de verificar **requisitos funcionales**
- Pruebas de aceptación de producción: llevadas a cabo por los futuros administradores de sistemas con el objetivo de verificar **requisitos de los administradores de sistemas** (normalmente no funcionales)
- Puede ser necesario establecer más niveles de pruebas. Por ejemplo, es frecuente incluir la evaluación de los productos intermedios dentro del plan maestro: revisión de documentos de requisitos, casos de uso, diseño funcional y técnico, actas de reuniones, ...

Distinción entre requisitos de usuario y requisitos funcionales

- Requisitos de usuario:
 - Condiciones o capacidades necesarias para que el usuario pueda resolver un problema o alcanzar un objetivo
 - Expresados en lenguaje natural y con poco nivel de detalle
- Requisitos de sistema:
 - Extensión de los requisitos de usuario expresados con mayor nivel de detalle y formalismo y desde el punto de vista del desarrollador
 - Clasificados habitualmente como funcionales o no funcionales
- También conocidos como *needs* y *requirements*
 - *Need: user needs to do something to user goal*
 - Ej: El usuario necesita entender bajo que condiciones puede utilizar un conjunto de datos.
 - *Requirement: entity / system should do something to enable users to do something*
 - Ej: El sistema deberá publicar el fichero con la licencia de uso del conjunto de datos en la página de descarga de los conjuntos de datos para que los usuarios puedan consultar bajo que condiciones se pueden utilizar.

- Método ...

- Determinar el nivel de detalle de las pruebas por cada característica/parte del sistema y nivel de pruebas

Característica /componente	Clase de riesgo	Pruebas de desarrollo	Pruebas de Sistema	Pruebas de aceptación	Evaluación	<Otro nivel de pruebas>
Funcionalidad						
- componente 1	A	●●●●	●●●●	●●●●		
- componente 2	A	●●●●	●●●●	●●●●		
- sistema global	B		●●	I		
Facilidad de uso	B	I	I	●●		
Rendimiento						
- en línea						
- por lotes						
Seguridad						
Idoneidad						

aclaración de valores en la tabla anterior:

●	Pruebas dinámicas con un nivel de detalle limitado
●●	Pruebas dinámicas con un nivel de detalle medio
●●●	Pruebas dinámicas con un nivel de detalle alto
S	Pruebas estáticas (inspección y revisión de los productos sin ejecución del software)
I	Pruebas implícitas (incluidas en otros tipos de pruebas sin diseñar/crear casos de prueba específicos)
<vacío>	Si una celda está vacía, significa que el nivel de pruebas o evaluación no está relacionado con la característica



- Productos
 - Sección del plan maestro (estrategia y resumen de cada nivel de pruebas)
- Técnicas (no aplicable)
- Herramientas (no aplicable)



3.1.5. Estimar el esfuerzo

- Propósito
 - Estimar el esfuerzo requerido
- Método
 - Ver más adelante técnicas de estimación del esfuerzo de las pruebas
- Productos
 - Sección del plan maestro
- Herramientas
 - Herramientas de planificación y monitorización de progreso
 - Ej: hojas de cálculo, Gantt project



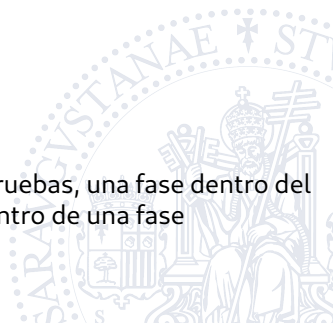
Productos: ejemplo en plan maestro

Fases del ciclo de vida de TMAP:
Planificación, Control,
Infraestructura, Preparación,
Especificación, Ejecución,
Finalización

Nivel de pruebas	Quien	PI	Co	In	Pr	Es	Ej	Fi	Total (horas)
Gestión global del proceso de pruebas	Gestor de las pruebas	10	5						15
Pruebas de sistema	Coordinador de las pruebas	5	5						10
	Especialistas de pruebas					10	65		75
Pruebas de aceptación	Gestor de pruebas								
	Especialistas de pruebas								
	Usuarios finales								
Pruebas de desarrollo	Coordinador de las pruebas	5	5						10
	Especialistas de pruebas				25	20			45
	Desarrollador						50	5	55
Totales:		20	15		25	30	105	5	200

Técnicas de estimación del esfuerzo de las pruebas

- Técnicas para la estimación del número de horas
 - Basada en el tamaño del objeto a probar
 - Estimación del esfuerzo de evaluación
 - Basada en la extrapolación
 - Basada en la Estructura de Descomposición de Trabajo
 - Análisis de los puntos de prueba
- Técnicas para la distribución del esfuerzo
 - Basada en ratios
 - Proporcional basada en los riesgos
- Se pueden aplicar a distintas escalas:
 - todo el proceso de pruebas, un nivel de pruebas, una fase dentro del ciclo de vida de un nivel, una actividad dentro de una fase



Estimación basada en el tamaño del objeto a probar

- Esta técnica se basa en dos elementos
 - Calcular el tamaño del proyecto en una unidad estándar (Test Object Size Meter – TOSM)
 - Contar con una tasa de número de horas de prueba por unidad (1.5-4 horas por TOSM)
- Esfuerzo de las pruebas = n° TOSM x horas de prueba por unidad
- Cada organización mantiene unas pautas para calcular el n° de TOSM de un proyecto
 - Basándose en puntos de función (ver asignatura de proyectos)
 - n° puntos función = n° TOSM
 - Basándose en el modelo de datos
 - < 10 entidades = 25 TOSM; 10-25 entidades=28 TOSM; > 25 entidades = 35 TOSM
 - Basándose en las páginas de documentos de requisitos
 - 1 página = 15 TOSM
 - Basándose en el número de pantallas
 - 1 pantalla = 8 TOSM
 - Basándose en el tamaño del código fuente de un sistema que se va a actualizar
 - 1 KLOC en C = 6.6 TOSM; 1 KLOC en Lisp = 16 TOSM

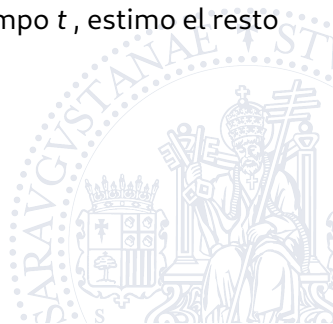
Estimación del esfuerzo de la evaluación (inspección, revisión, ...)

- Un evaluador tiene una tasa de 1-4 páginas por hora dependiendo de:
 - experiencia
 - , características de la calidad a comprobar
 - , detalle de la revisión exigida



Estimación basada en extrapolación

- Se basa en tomar medidas tan pronto como se inicia el proyecto
- Una vez que he acabado una actividad que representa el $X\%$ del tamaño del proyecto de pruebas en un tiempo t , estimo el resto de actividades según su tamaño



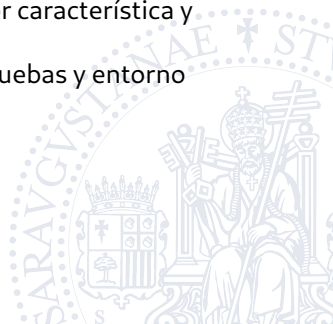
Estimación basada en la Estructura de Descomposición de Trabajo (*Work Breakdown Structure*)

- Se basa en descomponer las actividades de pruebas en actividades parciales hasta que seamos capaces de estimar el esfuerzo requerido
 - Se tiene también en cuenta el nivel de detalle de las pruebas que queremos realizar
- El esfuerzo total se calcula como la suma

Característica de la calidad	Estrategia	Horas
Facilidad de uso	●	70
Seguridad	● ● ●	80
Portabilidad	●	28
Total		178

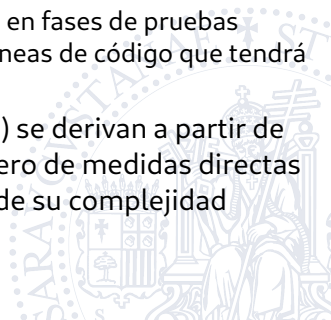
Característica de la calidad	Estrategia	Horas
Facilidad de uso	● ● ●	130
Seguridad	● ● ●	80
Portabilidad	●	28
Total		238

- *Test Point Analysis* = TPA
- Método paramétrico (basado en fórmulas) que tiene en cuenta
 - Tamaño (puntos de función)
 - Estrategia de pruebas (nivel de detalle por característica y componente)
 - Productividad: habilidad del equipo de pruebas y entorno
- Ver plantilla *Test Point Analysis* en TMAP



Créditos: asignatura Proyectos, DIIS, EINA

- Medida de la funcionalidad requerida por el sistema
- A partir de esta métrica para un sistema y de datos históricos de la misma en otros sistemas (lo bastante parecidos) podemos estimar
 - El coste de desarrollar el sistema
 - El número de errores que encontraremos en fases de pruebas
 - El número de componentes, módulos o líneas de código que tendrá el sistema
- Los puntos de función (function points, FP) se derivan a partir de una relación empírica entre un cierto número de medidas directas de base, y de las valoraciones cualitativas de su complejidad

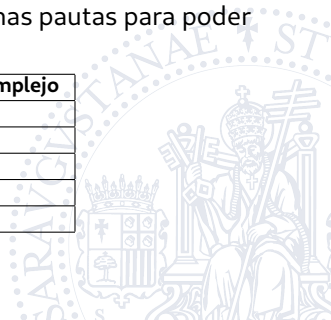


- Los valores de base son:

- Número de entradas (*Inputs*): cada una se origina en un usuario o en otra aplicación y generalmente resulta en una modificación de datos internos
- Número de salidas externas (*Outputs*): datos derivados dentro de la aplicación que proporcionan información al usuario (informes, pantallas, mensajes de error etc. No se contabilizan por separado los items de información que salen en un mismo informe o pantalla)
- Número de consultas externas (*Inquiries*): peticiones al sistema que devuelven alguna respuesta sin cambiar datos internos
- Número de ficheros lógicos internos (*Files*): agrupaciones lógicas de datos que el sistema debe mantener. Por ejemplo, si tenemos una BD con 1000 registros de personas, lo contaremos como uno, pero si la BD tiene datos de personas y de departamentos de la empresa, normalmente lo contaremos como dos
- Número de ficheros de interfaz externa (*Interfaces*): agrupaciones lógicas de datos externas al sistema pero que se usan (p.ej., un fichero de datos que se comparte entre dos aplicaciones contaría como interfaz para las dos)

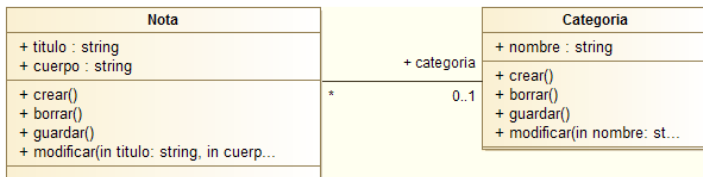
- Hay que identificar los elementos de la transparencia anterior, contarlos, aplicarles un peso según los consideremos simples, promedio o complejos y sumar el total. A esto se le llama **puntos de función sin ajustar**
 - El método no da reglas claras para contar puntos de función (aunque hay bastante literatura posterior sobre el tema). Lo mínimo es tener siempre presente que las cosas que vayan a costar más esfuerzo de desarrollo deben tener asignados más puntos
 - Y ser consistentes: usar siempre las mismas pautas para poder aprovechar datos históricos y experiencia

	Simple	Medio	Complejo
Outputs	4	5	7
Inquiries	3	4	6
Inputs	3	4	6
Files	7	10	15
Interfaces	5	7	10

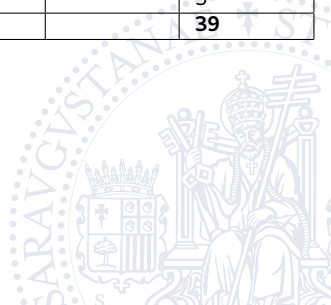


Ejemplo para una aplicación de gestión de notas en dispositivos móviles

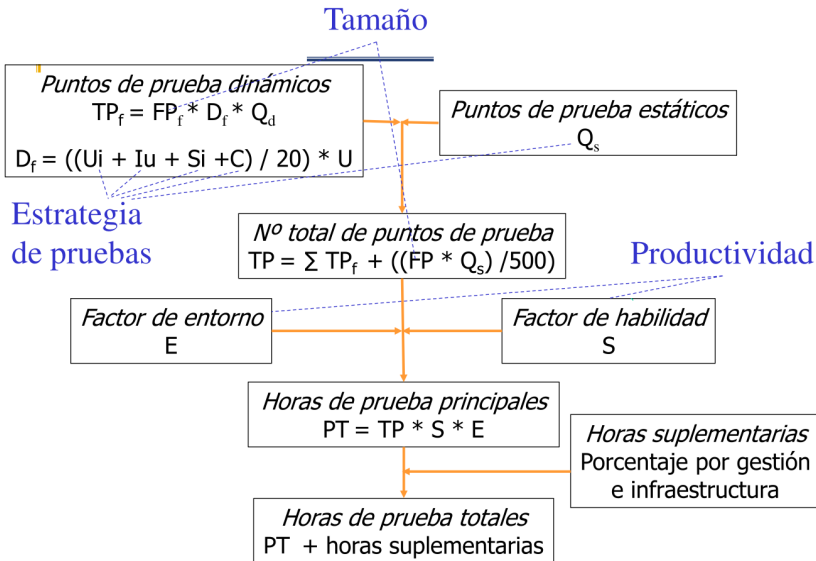
- Aplicación similar a la incluida como ejemplo dentro de la práctica 2 de Ingeniería del Software
- Se pretende estimar el proceso de pruebas sobre una modificación de la aplicación para añadir la siguiente funcionalidad:
 - Asociar una categoría a una nota
 - Listar y gestionar las categorías disponibles
 - Ordenar notas por categoría o título
 - Filtrar las notas por categoría
 - Enviar notas (por e-mail o SMS)



Tipo	Simples	Medios	Complejos	Totales
Inputs	Edición de notas, Edición de categorías			$2 \times 3 = 6$
Outputs	Lista de notas, Lista de categorías			$2 \times 4 = 8$
Inquiries	Ordenación de notas, Filtrado por categorías			$2 \times 3 = 6$
Files	Datos de notas, Datos de categorías			$2 \times 7 = 14$
Interfaces	Envío de notas			5
Total				39



Vista general: estimación basada en TPA



Puntos de entrada: Puntos de función (FP_f)

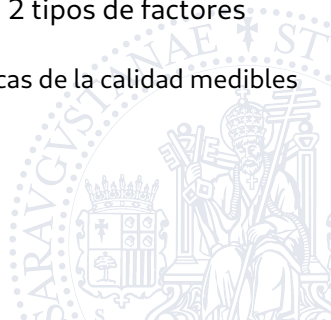
- En la métrica de puntos de función se analizan los requisitos funcionales para ver si corresponden a alguno de los 5 tipos establecidos: *inputs, outputs, inquiries, files, interfaces*
- Para aplicar TPA debemos reagrupar los puntos de función según las funcionalidades facilitadas a usuarios finales (FP_f)
 - Los puntos de un file se añaden a los puntos correspondientes al *input* que origina su entrada
 - Los puntos de una *interface* se añaden a la función que la maneja
 - Una funcionalidad final que es prácticamente una replica de otra ya existente tiene los mismos puntos que la original. La clase que implementa esta funcionalidad se denomina clase *clone*.
 - Si es posible, también se asignan puntos a funcionalidades que no tienen que ser especificadas y/o desarrolladas pero si incorporadas a través de clases denominadas *dummy* y probadas en el proyecto

Ejemplo para la aplicación de gestión de notas

- Puntos de función como entrada para TPA
 - Gestión de notas
 - *input + output + file + inquiries*
 - $3 + 4 + 7 + 2 \times 3 = 20$
 - Gestión de categorías
 - *input + output + file*
 - $3 + 4 + 7 = 14$
 - Envío de notas
 - *inputs (actividad de envío de Mail/SMS de Android como función dummy) + interface*
 - $3 + 5 = 8$



- Son la suma de los puntos de prueba por función en relación a las características de la calidad medibles
 - $TP_f = FP_f \times D_f \times Q_d$
- El número de puntos de prueba se basa en 2 tipos de factores
 - Factores dependientes de la función (D_f)
 - Factores que representan las características de la calidad medibles dinámicamente (Q_d)



Factores dependientes de la función - D_f

- $D_f = ((U_i + I_u + S_i + C)/20) \times U$

Factor	Pesos		
	Bajo	Medio	Alto
Importancia para el usuario (<i>User importance – U_i</i>)	3	6	12
Intensidad de uso (<i>Intensity of Use – I_u</i>)	2	4	8
Impacto en el sistema (<i>System Impact – S_i</i>)	2	4	8
Complejidad (<i>Complexity – C</i>)	3	6	12

Factor	Pesos	Función normal
	Clases <i>clone</i> , clases <i>dummy</i> o funciones raramente utilizadas	
Uniformidad (<i>Uniformity</i>)	0,6	1

- Impacto en el sistema (nivel en el que una modificación de una función tiene impacto en el sistema)
 - Se mide en base al nº de colecciones de datos sobre las cuales la función realiza modificaciones, y el nº de funciones que acceden a una misma colección o conjunto de colecciones

nº colecciones de datos	Funciones		
	1	2-5	>5
1	Bajo	Bajo	Medio
2-5	Bajo	Medio	Alto
>5	Medio	Alto	Alto

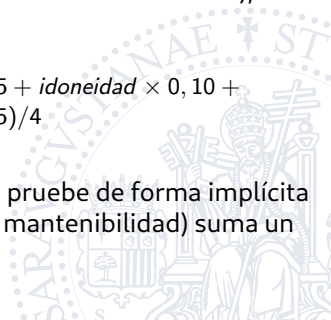
- Complejidad (nivel de complejidad del código)
 - Bajo: nº de nodos predicado ≤ 5
 - Medio: $6 \leq \text{nº nodos predicado} \leq 11$
 - Alto: nº nodos predicado > 11



Características de calidad medibles dinámicamente -

Q_d

- Mediante pruebas dinámicas explícitas
 - Solo se consideran medibles de forma explícita las siguientes
 - Funcionalidad, seguridad, idoneidad de infraestructura, rendimiento, portabilidad
 - Pesos posibles: 0 (no importante), 3 (requisitos de calidad bajos), 4 (requisitos de calidad normales), 5 (requisitos de calidad altos), 6 (requisitos de calidad muy altos)
 - Peso de la parte explícita
 - $(funcionalidad \times 0,75 + seguridad \times 0,05 + idoneidad \times 0,10 + rendimiento \times 0,05 + portabilidad \times 0,05)/4$
- Mediante pruebas dinámicas implícitas
 - Cualquier característica de calidad que se pruebe de forma implícita (facilidad de uso, eficiencia, rendimiento, mantenibilidad) suma un 0,02 adicional
- $Q_d = parte\ explicita + parte\ implicita$

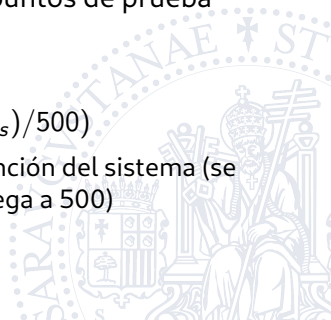


Nº total de puntos de prueba (TP)

- Aparte de los puntos de función dinámicos (TP_f), se consideran también las características de calidad probadas de forma estática (mediante evaluaciones)
- Cada característica de calidad probada de forma estática tiene un peso de 16 que se suma para calcular los puntos de prueba estáticos, denominados Q_s
- Nº total de puntos de prueba:

$$TP = \sum TP_f + ((FP \times Q_s)/500)$$

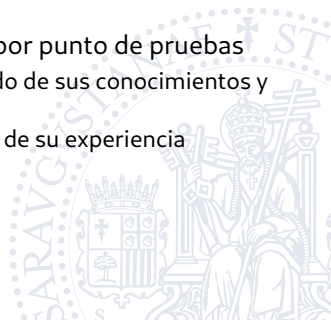
- FP representa el nº total de puntos de función del sistema (se asigna un valor mínimo de 500 si no se llega a 500)



- Los puntos de prueba se modulan teniendo en cuenta la habilidad del equipo de pruebas y el entorno de las pruebas

$$PT = TP \times S \times E$$

- Factor de habilidad (*skill factor* - S)
 - Se define como el nº de horas requerido por punto de pruebas
 - 1-2 horas para un probador, dependiendo de sus conocimientos y habilidades
 - 2-4 horas para un usuario, dependiendo de su experiencia



- Factor de entorno - E

- El entorno de las pruebas también juega un papel relevante como facilitador del proceso de pruebas

Factor	Pesos		
	Bajo (se dispone)	Medio (hay algo)	Alto (no hay nada)
Herramientas de prueba	1	2	4
Proceso de pruebas previo	2	4	8
Base de las pruebas	3	6	12
Entorno de desarrollo	2	4	8
Entorno de pruebas	1	2	4
Documentación del proceso de pruebas (<i>testware</i>)	1	2	4

- E = suma de los pesos para cada factor dividida por un valor nominal (21)

Horas de prueba totales

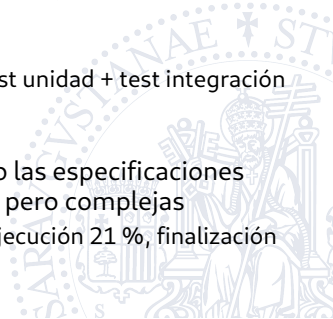
- Todo proceso de pruebas requiere horas suplementarias para tareas de gestión y mantenimiento de infraestructura que se deberían contabilizar
 - Algunas organización aplican un porcentaje fijo adicional (entre el 5 % y el 20 %)
 - En otras organizaciones se utiliza un porcentaje por defecto (12 %) que se regula según las características de la organización

Factor	Ampliaciones o reducciones del porcentaje		
	Reduce	Neutro	Aumenta
Nº personas	<=4	5-9	>=10
Nº herramientas de gestión	>=3	<3	0
Existencia de una organización permanente de pruebas	Existe		No existe

- Horas de prueba totales = PT + horas suplementarias

Estimación basada en ratios

- Es una estimación basada en la experiencia de anteriores proyectos
- El esfuerzo de las pruebas se mide con porcentajes en relación al esfuerzo de desarrollo
- Ejemplos en base a la experiencia de la empresa XX:
 - Utilizando un ciclo de vida en cascada y un lenguaje estructurado (C, Ada)
 - Análisis + Diseño de sistemas -> 20 %
 - Diseño detallado + implementación + test unidad + test integración -> 50 %
 - Pruebas de sistema -> 30 %
 - Dentro de las pruebas de sistema, cuando las especificaciones proporcionadas por el cliente son buenas pero complejas
 - Preparación 6 %, especificación 54 %, ejecución 21 %, finalización 2 %, control e infraestructura 17 %



Estimación proporcional basada en los riesgos

- Se toma como base el presupuesto asignado para pruebas, y se reparte el presupuesto en función del tamaño de componentes, y su clasificación de riesgo
 - Factores de dedicación según clase de riesgo
 - clase A -> factor 1.5; clase B -> factor 1; clase C -> factor 0.6

Componente	Tamaño	Clase de riesgo	Factor	Tamaño x Factor	Nº horas
1	10	C	0.6	6	7.86
2	15	A	1.5	22.5	29.48
3	7	B	1	7	9.17
4	25	A	1.5	37.5	49.12
5	5	C	0.6	3	3.93
Total				76	100

Presupuesto: 100 horas

Escalado: presupuesto / tamaño teniendo en cuenta riesgo -> $100/76 = 1.31$

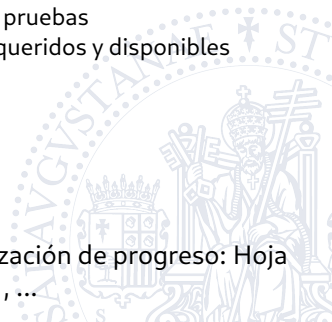
Ejercicios

- 1 Utiliza las técnicas de estimación basadas en el tamaño y la técnica de análisis de puntos de prueba para estimar el número de horas del proceso de pruebas del trabajo de la asignatura
 - Ver apartado 10.1 de la plantilla del Plan de Pruebas Maestro
- 2 Utiliza las técnicas de estimación orientadas a la distribución de esfuerzos para estimar el número de horas por actividad del trabajo de la asignatura
 - Ver apartado 10.1 de la plantilla del Plan de Pruebas Maestro
- 3 Dada la siguiente tabla donde se indica el tamaño de los componentes de un sistema y su clase de riesgo, ¿Cuál sería el número de horas dedicadas a las pruebas de cada componente utilizando una estimación proporcional basada en riesgos si disponemos de 90 horas para dedicar en total al proceso de pruebas y el factor de dedicación para las clases de riesgo A, B y C es 1,5, 1 y 0,5 respectivamente?

Componente	Tamaño	Clase de riesgo	Horas
1	10	A	
2	20	B	
3	20	C	

3.1.6. Determinar la planificación

- Propósito
 - Realizar la planificación global
- Método
 - La planificación debería incluir
 - Actividades a ejecutar a nivel de fase por cada nivel de pruebas
 - Relaciones y dependencias con otras actividades
 - Tiempo a dedicar para cada nivel de pruebas
 - Tiempo disponible para llevar a cabo las pruebas
 - Recursos (personal e infraestructura) requeridos y disponibles
- Entregables
- Productos (sección del plan maestro)
- Técnicas (no aplicable)
- Herramientas
 - Herramientas de planificación y monitorización de progreso: Hoja de cálculo, Gantt, MSPProject , Projektron , ...



Productos: ejemplo de sección en plan maestro

		Nov	Dic	Ene	Feb	Mar	Abr	May
1	Planificación (plan maestro)							
2	Gestión							
3	Pruebas de desarrollo							
4	Planificación							
5	Preparación, especificación							
6	Ejecución, finalización							
7	Pruebas de sistema							
8	Planificación							
9	Preparación, especificación							
10	Ejecución, finalización							
11	Pruebas de aceptación							

El detalle de las actividades a realizar se muestra en la siguiente tabla:

Nr	Actividad	Fecha de inicio	Fecha de fin	Duración	Dependencias
1	Planificación	1/11/2011	30/11/2011	1 mes	
2	Gestión	1/12/2011	31/5/2012	6 meses	1
...					

3.1.7. Definir los productos de las pruebas

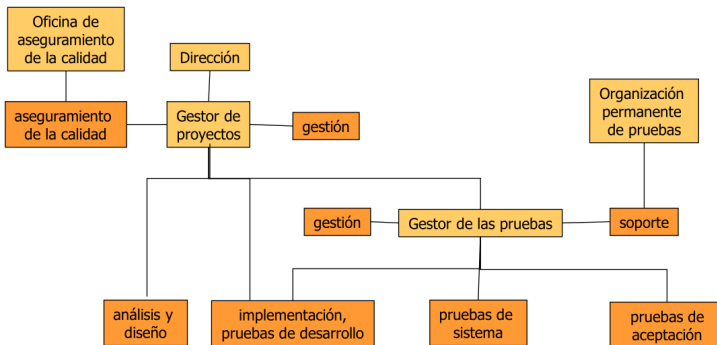
- Propósito
 - Definir los productos de las pruebas que se generan a nivel de plan maestro y a través de los distintos niveles
- Método
 - Ejemplos de especificación de productos a entregar:
 - Plan maestro, informes, procedimientos, instrucciones, documentación de proyecto
 - También se puede especificar los productos de los niveles de pruebas: planes, scripts, documentación de las pruebas, estándares
- Productos
 - Especificación de los productos a entregar a nivel general y por cada nivel de pruebas
- Técnicas (no aplicable)
- Herramientas (no aplicable)
 - Aunque es probable que se haga referencia a posibles herramientas para dar soporte a la gestión de documentación de pruebas (Alfresco , Google Drive, OneDrive, ...)

Productos: ejemplo de sección en plan maestro

Fase	Producto	Comentario	Fecha de entrega
Planificación	Plan de pruebas maestro Plan de pruebas de desarrollo Plan de pruebas de sistema Plan de pruebas de aceptación		
Control/gestión	Informe de riesgos		
Infraestructura	Especificación detallada del entorno de pruebas		
Preparación	Informe sobre la capacidad de prueba del sistema		
Especificación	Script de pruebas		
Ejecución	Registro de defectos		
Finalización	Informe final de pruebas Consejo sobre lanzamiento de versión		

3.1.8. Definir la organización

- Propósito
 - Definir los roles, tareas, autorizaciones y responsabilidades para el proceso global a través de los niveles de pruebas
- Método
 - Describir la organización

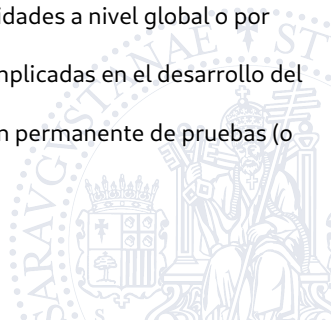


Sobre la organización

- Es conveniente tener una política general de pruebas
 - En algunas situaciones es más eficiente organizar ciertos asuntos de forma centralizada que por proyecto
 - Establecer cómo gestionar a las personas, los recursos y los métodos implicados en los procesos de pruebas en línea con las políticas estratégicas, tácticas y operacionales de la organización
- Si la empresa de desarrollo tiene recursos suficientes, sería conveniente contar con una organización permanente de pruebas que trabaja de forma horizontal en todos los proyectos
 - Ventajas: mejor aprovechamiento de los expertos en pruebas, estandarización de los productos de pruebas, minimizar el tiempo de arranque, mejora continua,
 - Tipos de organizaciones permanentes: centro de expertos (consultas puntuales), factoría de expertos (subcontratación de expertos para un proyecto)

- Método ...

- Determinar roles requeridos
 - Gestión/coordinación de las pruebas
 - Otros tipos de gestión: gestión de la infraestructura, administración de defectos en los niveles de prueba
 - Soporte: expertos de pruebas, expertos de dominio y desarrolladores
- Asignar tareas, autorizaciones y responsabilidades
 - Definición de roles, tareas y responsabilidades a nivel global o por niveles de pruebas
 - Establecer relaciones con otras partes implicadas en el desarrollo del sistema
 - Establecer relaciones con la organización permanente de pruebas (o depto. calidad)



- Método ...
 - Asignar personal

Rol	Departamento / Nombre del empleado(s)	No. horas por semana	Periodo	Descripción de tareas y responsabilidades
Gestor de las pruebas				Escribe el plan de pruebas maestro Coordina el proceso general de pruebas
Coordinador de las pruebas				Escribe el plan para un nivel de pruebas Coordina pruebas en ese nivel
Especialista de pruebas				Especifica pruebas (Re)ejecución de pruebas
Administrador de sistemas				Configuración del entorno de pruebas

- Método ...
 - Establecer requisitos de formación y *coaching*
 - Los profesionales de las pruebas deben poseer conocimiento sobre pruebas, la infraestructura y dominio de la aplicación a probar
 - Otras habilidades deseables: buena comunicación oral y escrita; precisión y capacidades analíticas; convincente y perseverante; realista y con una actitud crítica positiva; creativo
 - Establecer estructuras de consulta e informes
 - Ver ejemplo en siguiente página
- Productos (sección en plan maestro)
- Técnicas (no aplicable)
- Herramientas (no aplicable)



Ejemplo de estructuras de consulta e informes

Tipo	Objetivo	Frecuencia	Quien
Reunión de proyecto	Discusión del progreso general del proyecto	Semanal	Gestor del proyecto, Gestor de las pruebas
Reunión de progreso para cada nivel de pruebas	Discusión del progreso de cada nivel de pruebas	Semanal	Gestor de las pruebas, Especialistas de pruebas
Revisión de defectos	Discusión y establecimiento de la prioridad de los defectos encontrados	Semanal	Gestor del proyecto, Gestor de las pruebas, Analista/Programador

Tipo	Objetivo	Frecuencia	Quien
Informe de riesgos	Detallar los riesgos identificados de las pruebas	Ad-hoc / Bajo demanda	Equipo de pruebas
Consejo sobre lanzamiento de versión	Aconsejar sobre calidad/riesgos del objeto de pruebas	Una sola vez	Gestor del proyecto
Informe final de pruebas	Detallar la evaluación del proceso de pruebas y del objeto de las pruebas	Una sola vez	Gestor del proyecto
Informe de progreso	Detallar progreso de las pruebas y calidad/riesgos del objeto de pruebas	Semanalmente	Gestor de las pruebas a los clientes, gerentes, ...
Informe de defectos	Detallar los defectos y su estado	Semanalmente	Gestor de las pruebas a los clientes, gerentes, ...

3.1.9. Definir la infraestructura

- Propósito
 - Determinar la infraestructura requerida para el proceso de pruebas en una fase temprana, en particular las partes que se deben establecer para varios niveles de pruebas o a largo plazo
- Método
 - Definir el entorno de pruebas
 - hardware, software, datos, ... a utilizar en la ejecución de las pruebas
 - Definir las herramientas
 - Herramientas de prueba para automatizar la planificación, el control, la especificación y la ejecución de las pruebas
 - Beneficios: Incremento de productividad, mejora en la calidad de las pruebas, las pruebas resultan más gratificantes, más opciones de prueba

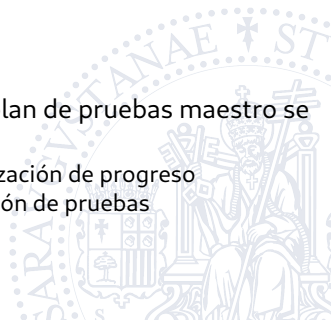
- Método ...
 - Definir la infraestructura asociada al lugar de trabajo
 - Garantizar la disponibilidad de un lugar de trabajo donde los profesionales de las pruebas puedan realizar su trabajo en buenas condiciones, con eficiencia y efectividad
 - Establecer la planificación de la infraestructura
- Productos (sección en el plan maestro)
- Técnicas
 - Lista de comprobación de necesidades típicas relacionadas el lugar de trabajo (*ver Office setup en TMAP*)
 - Lista de comprobación de necesidades típicas relacionadas con el entorno de las pruebas (*ver Test environment en TMAP*)
- Herramientas (no aplicable)



3.1.10. Organizar la gestión

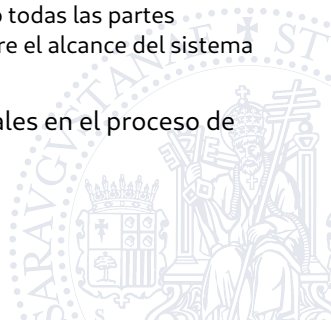
- Propósito
 - Establecer aspectos centralizados de la gestión del proceso de pruebas, infraestructura, productos y defectos
- Método
 - Definir la gestión del proceso
 - Indicar si es necesario registrar información sobre progreso y uso del presupuesto y tiempo
 - Indicar si es necesario registrar información sobre indicadores de calidad, estadísticas de las pruebas
 - Definir la gestión de la infraestructura
 - Indicar si es necesario gestionar entorno/herramientas compartidas entre niveles de pruebas
 - Indicar si es necesario alinear planificaciones de instalación de infraestructura
 - Indicar si es necesario actuar como intermediario entre la organización permanente de pruebas y los niveles de pruebas

- Método ...
 - Definir la gestión de productos de pruebas
 - Indicar los estándares a seguir para los productos de pruebas de los distintos niveles
 - Definir el procedimiento de gestión de defectos
 - Ver más adelante sección 4.6
- Productos
 - Sección del plan maestro
- Técnicas (no aplicable)
- Herramientas (no aplicable)
 - Aunque es posible que en la sección del plan de pruebas maestro se haga referencia a:
 - Herramienta de planificación y monitorización de progreso
 - Herramienta de gestión de documentación de pruebas
 - Herramienta de gestión de defectos



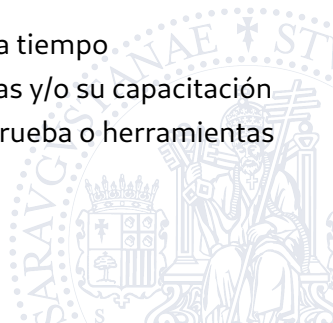
3.1.11. Determinar los riesgos del proceso de pruebas y las contingencias

- Propósito
 - Determinar de forma explícita los riesgos del proceso de pruebas
- Método
 - Determinar riesgos y las medidas a tomar para cubrir/mitigar el riesgo
 - Ejemplos de riesgos: No han intervenido todas las partes involucradas, discusiones continuas sobre el alcance del sistema
- Técnicas
 - Lista de comprobación de riesgos habituales en el proceso de pruebas
- Productos
 - Sección del plan maestro
- Herramientas (no aplicable)



Lista de comprobación de riesgos habituales en el proceso de pruebas

- Ausencia de planificación en el equipo de desarrollo
 - No se sabe cuando dispondremos del objeto de pruebas
- El plan de pruebas depende de los planes de otros actores del proyecto
- La base de las pruebas no está disponible a tiempo
- Falta de recursos humanos para las pruebas y/o su capacitación
- Falta de disponibilidad de un entorno de prueba o herramientas suficientes
- ...

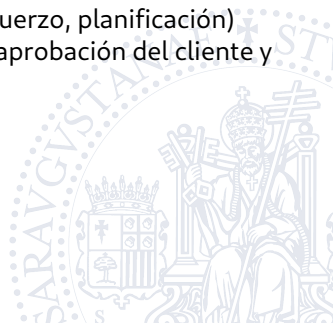


Productos: ejemplo de sección en plan maestro

Nº	Evento	Consecuencia	Impacto	Probabilidad	Puntuación	Medidas	Propietario (Responsable de tomar medidas)
1	No intervienen todas las partes involucradas. El cliente es una organización que agrupa a varios miembros.	Es probable que los miembros de la organización no reciban algunas notificaciones/documentos donde se especifican pruebas de aceptación.	Medio	100 %	8	Se solicita al representante de la organización cliente que establezca una lista de correo, y comunique la máxima información posible a los miembros	Representante del cliente
2	El cliente tiene escasa formación sobre procesos de prueba	Las pruebas de aceptación se realizan de forma intuitiva y exploratoria	Medio	50%	6	Las pruebas de sistema deberían ser exhaustivas, y el cliente solo se concentra en pruebas de aceptación de la funcionalidad añadida	Gestor del proyecto
3	No está claro el alcance del proyecto		Alto	70%	10	El proyecto se desarrolla en dos iteraciones para que los clientes tengan pronta visibilidad	Gestor del proyecto

3.1.12. Realimentar y consolidar plan

- Propósito
 - Registrar los resultados de todas las actividades ejecutadas hasta la fecha
 - Obtener aprobación del cliente para la aproximación seleccionada
- Método
 - Crear el plan maestro
 - Reajustar el plan maestro (estrategia, esfuerzo, planificación)
 - Consolidar el plan maestro (conseguir la aprobación del cliente y otras partes implicadas)
- Productos
 - el plan maestro
- Técnicas (no aplicable)
- Herramientas (no aplicable)



Productos: ejemplo de modificación en plan maestro

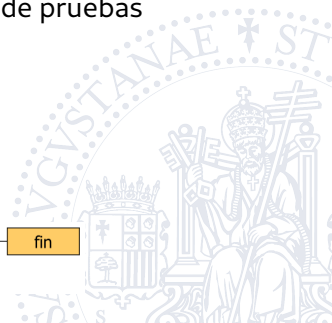
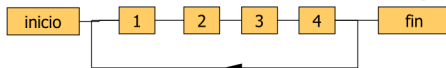
Característica /componente	Clase de riesgo	Pruebas de desarrollo	Pruebas de sistema	Pruebas de aceptación	Otro nivel de pruebas
Funcionalidad					
- Subsistema 1	A	● 0	● ● ● -	● ●	

Aligero las pruebas

Refuerzo las pruebas

3.2. Control

- El propósito es controlar el proceso de pruebas, la infraestructura y los productos de las pruebas a nivel general para proporcionar información continua sobre el progreso y la calidad del proceso de pruebas
- En conformidad con la frecuencia y formas definidas en el plan de pruebas, se realizan informes sobre la calidad de los objetos a probar y el progreso y calidad del proceso de pruebas
- Actividades:
 - 1 Gestionar
 - 2 Monitorizar
 - 3 Informar
 - 4 Reajustar



3.2.1. Gestionar

- Propósito
 - Gestionar el proceso de pruebas, la infraestructura y los productos de pruebas a nivel global para proporcionar información continua del progreso y calidad del proceso de pruebas global
- Método
 - Gestión del proceso de pruebas, infraestructura, productos de pruebas y defectos
 - Verificar la implementación de los estándares de gestión generales
- Productos (un proceso de pruebas gestionado)
- Técnicas (no aplicable)
- Herramientas
 - Herramienta de planificación y monitorización de progreso
 - Herramienta de gestión de documentación de pruebas
 - Herramienta de gestión de defectos

3.2.2. Monitorizar

- Propósito
 - Utilizando información interna y externa monitorizar el proceso global de pruebas
- Método
 - Monitorizar eventos/desviaciones sobre la planificación prevista
 - Ejemplos: un nivel de pruebas se desvía de la planificación, un nivel de pruebas no tiene recursos para ejecutar las pruebas requeridas, incertidumbre acerca del alcance de los niveles de pruebas, un gran número de defectos para una característica/parte del sistema
 - Para cada evento, se indican: posibles causas, consecuencias para el proceso global de pruebas, posibles medidas a tomar
- Productos (medidas de control propuestas)
- Técnicas (no aplicable)
- Herramientas
 - Herramienta de planificación y monitorización de progreso
 - Herramienta de gestión de documentación de pruebas
 - Herramienta de gestión de defectos

3.2.3. Informar

- Propósito
 - Creación de informes para detallar la calidad del objeto de pruebas y el progreso y calidad de los niveles de pruebas y del proceso global de pruebas
- Método
 - Tipos de informes a realizar: informe de progreso, informe de riesgos, consejos de lanzamiento de release, informe final
- Productos
 - Informes, análisis coste-beneficio
- Técnicas
 - Lista de comprobación de problemas típicos que pueden surgir durante el proceso de pruebas (*ver Test process evaluation en TMAP*)
 - Elaboración de métricas (ver siguiente página)
- Herramientas
 - Herramienta de planificación y monitorización de progreso
 - Herramienta de gestión de documentación de pruebas
 - Herramienta de gestión de defectos

Elaboración de métricas

- La definición, mantenimiento y uso de métricas es importante para el proceso de pruebas porque permite que el gestor de pruebas pueda contestar (con datos) a cuestiones como:
 - ¿Cuál es la calidad de los componentes probados?
 - ¿Cómo va el progreso del proceso de pruebas?
- Una posible aproximación para definir/realizar un conjunto de métricas es el método Objetivo-Cuestión-Métrica (*Goal-Question-Metric*):

- 1 Definir los objetivos:
 - **conocer** detalles sobre el **objeto de pruebas** (¿hemos sido **efectivos**?)
 - **mejorar** el **proceso** de pruebas (¿podemos mejorar la **eficiencia** – ej: reducir esfuerzo de pruebas en 20 % –?)
- 2 Hacer preguntas por objetivo
- 3 Extraer las métricas a partir de las preguntas
- 4 Recolección de datos y análisis (utilizar formularios electrónicos a ser posible)
- 5 Presentación y distribución de los datos medidos
- 6 Relacionar los datos con las cuestiones y objetivos
 - Establecer indicadores para ver si se han conseguido objetivos

● Ejemplo de GQM

Objetivo: proporcionar detalle de la calidad del componente a probar

Cuestiones

¿Cuántos defectos se han encontrado durante el proceso de pruebas?

¿Cuántos defectos se han encontrado en producción? (hasta 3 meses después del inicio de producción)

¿Cuál es el grado de cobertura de las pruebas?

etc.

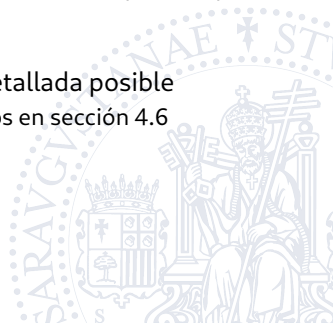
Medidas

Número total de defectos

Número de defectos encontrados por usuarios después del inicio de producción

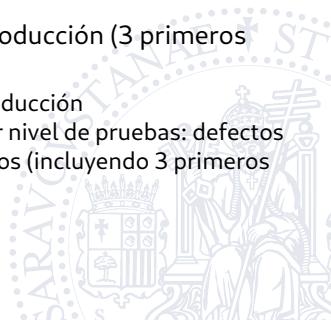
Ratio entre número de caminos del programa cubiertos por la ejecución de las pruebas y el número total de caminos del programa

- Registros de datos necesarios para el cálculo de métricas:
 - Para cada persona realizando pruebas:
 - registro de horas por actividad de pruebas
 - Para cada producto/entregable de las pruebas:
 - nombre de documento, fecha de entrega, fase de pruebas, versión, atributo acerca de cantidad (nº páginas, nº casos de pruebas por script de pruebas, ...)
 - Hojas de control de ejecución de pruebas
 - Registrar los defectos de la forma más detallada posible
 - ver procedimiento de gestión de defectos en sección 4.6

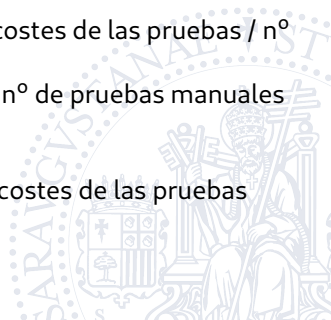


- Métricas orientadas a medir la **efectividad**:

- Ratio de nº de defectos encontrados: $\text{nº de defectos encontrados} / \text{tamaño del sistema (en base a puntos de función o KLOC)}$
- Ratio de instrucciones ejecutadas: $\text{nº instrucciones probadas} / \text{nº total de instrucciones}$
- Ratio de nº de pruebas: $\text{nº de pruebas} / \text{tamaño del sistema}$
- Ratio de nº de caminos probados: $\text{nº de caminos lógicos probados} / \text{nº de caminos lógicos presentes}$
- Si conocemos también los defectos en producción (3 primeros meses)
 - Nº de defectos encontrados durante producción
 - Efectividad de detección de defectos por nivel de pruebas: $\text{defectos encontrados en nivel} / \text{nº total de defectos (incluyendo 3 primeros meses)}$

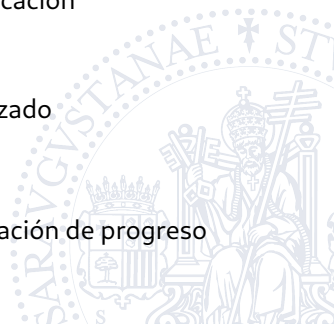


- Métricas orientadas a medir la **eficiencia** (productividad):
 - N° de defectos por hora de pruebas (y hora de ejecución de pruebas)
 - N° de casos de prueba por hora
 - N° de scripts especificados por hora
 - N° de defectos por script de pruebas
 - Ratio de horas dedicadas a cada fase de pruebas
 - Ratio de costes de pruebas: $\text{costes de las pruebas} / \text{coste total de desarrollo}$
 - Ratio de costes por defecto encontrado: $\text{costes de las pruebas} / \text{n° de defectos encontrados}$
 - Grado de automatización de las pruebas: $\text{n° de pruebas manuales} / \text{n° de pruebas automáticas}$
 - Ahorro de las pruebas: daño evitado
 - Beneficio de las pruebas: daño evitado – costes de las pruebas



3.2.4. Reajustar

- Propósito
 - Ajustar el proceso global de pruebas si es necesario y consultando con el cliente
- Método
 - Implementar medidas
 - Ajustar los productos de la fase de planificación
 - Comunicar al cliente los reajustes
- Productos
 - Medidas de control, plan maestro actualizado
- Técnicas (no aplicable)
- Herramientas
 - Herramienta de planificación y monitorización de progreso



1. Conceptos básicos

2. Fundamentos de TMAP

3. Gestión global del proceso de pruebas

4. Pruebas de sistema y aceptación

4.1 Planificación

4.2 Control

4.3 Instalación y mantenimiento de la infraestructura

4.4 Preparación

4.5 Especificación de casos de prueba

4.6 Ejecución

4.7 Finalización

5. Pruebas de desarrollo



4. Pruebas de sistema y aceptación

- Se considera que son procesos autónomos que se deben organizar
 - Tienen su propio plan, presupuesto y entorno de pruebas
- Son procesos autónomos que se ejecutan en paralelo al proceso de desarrollo y que comienzan al crearse las especificaciones funcionales
- Siguen las 7 fases del ciclo de vida de TMAP
 - 1 Planificación
 - 2 Control
 - 3 Instalación y mantenimiento de la infraestructura
 - 4 Preparación
 - 5 Especificación de casos de prueba
 - 6 Ejecución
 - 7 Finalización

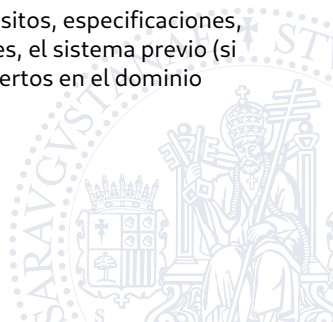


4.1. Planificación

- Se establece un plan de pruebas por nivel (con su propia estimación y recursos)
 - *Ver plantilla [Template test plan system and acceptance tests](#)*
- Actividades
 - 1 Formular la misión
 - 2 Entender la misión
 - 3 Determinar la base de las pruebas (**¡nueva!**)
 - 4 Analizar los riesgos del producto
 - 5 Determinar la estrategia de pruebas
 - 6 Estimar el esfuerzo
 - 7 Determinar planificación
 - 8 Asignar unidades y técnicas de pruebas (**¡nueva!**)
 - 9 Definir los productos de pruebas
 - 10 Definir la organización
 - 11 Definir la infraestructura
 - 12 Organizar la gestión
 - 13 Determinar los riesgos del proceso de pruebas y las contingencias
 - 14 Realimentar y consolidar plan

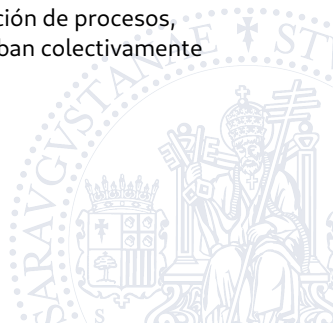
4.1.3. Determinar la base de las pruebas (¡nueva!)

- Propósito
 - Definir la base de las pruebas para que se conozcan lo antes posible los elementos con los que comparar los objetos de las pruebas
- Método
 - Definir e identificar la base de las pruebas
 - Recopilar los requisitos escritos y no escritos
 - Utilizar como base documentos de requisitos, especificaciones, manuales, entrevistas, actas de reuniones, el sistema previo (si existe), prototipos, conocimiento de expertos en el dominio
- Productos
 - sección en el plan de pruebas
- Técnicas (no aplicable)
- Herramientas (no aplicable)



4.1.8. Asignar unidades y técnicas de pruebas (¡nueva!)

- Propósito
 - Determinar los tipos de prueba y las técnicas para cada característica/componente en función del presupuesto, planificación y estrategia de pruebas
- Método
 - Determinar las unidades de pruebas
 - Unidad de prueba(*test unit – tu*) = colección de procesos, transacciones y/o funciones que se prueban colectivamente
 - Asignar las técnicas de prueba
- Productos
 - sección en el plan de pruebas
- Técnicas (no aplicable)
- Herramientas (no aplicable)



Algunas definiciones relacionadas con técnicas de pruebas

- Situación de prueba
 - Una condición aislada bajo la cual un objeto a probar muestra un comportamiento específico
- Caso de pruebas
 - Permite comprobar si el sistema muestra el comportamiento deseado bajo circunstancias específicas
 - Componentes: situación inicial, acciones a ejecutar, resultado esperado
- Caso de pruebas lógico
 - Describe en términos lógicos las circunstancias bajo las cuales se examina el sistema indicando las situaciones de prueba cubiertas por el caso de pruebas
- Caso de pruebas físico
 - Es la elaboración concreta de un caso de pruebas lógico, realizando una selección de los valores requeridos para las entradas y la configuración del entorno

- Script de pruebas
 - programas automáticos o procedimientos manuales que facilitan la ejecución de uno o varios casos de prueba de forma simple y eficiente
- Cobertura
 - Relación (ratio) entre lo que se puede probar y lo que se prueba realmente
- Tipo de cobertura
 - La forma utilizada para cubrir las situaciones de prueba a partir de la base de las pruebas (*test basis*)
 - Ejemplos de tipos de cobertura: caminos, clases (particiones) de equivalencia, análisis de los valores límite, condiciones/decisiones
- Técnica de pruebas (o técnica de diseño de pruebas)
 - Técnica que permite generar los casos de pruebas que consiguen el tipo de cobertura requerida a partir de la base de las pruebas
 - Cada tipo de cobertura exige una información específica en la base de las pruebas
 - Ej : cobertura de caminos -> diagrama de flujo o pseudo algoritmo
 - Las técnicas pueden estar relacionadas también con la característica de la calidad que se quiere comprobar
 - Ej : rendimiento -> simulación estadística de datos en entorno real (*Real Life Test*)

Cada organización puede acordar una serie de técnicas por característica e intensidad

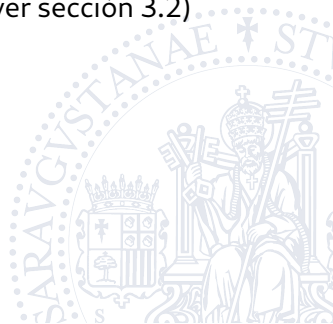
Característica de calidad	Técnica de diseño de pruebas (intensidad)		
	Ligera	Media	Alta
Funcionalidad	Valores típicos de error Valores imposibles Pruebas exploratorias (diseño de casos durante ejecución)	Clases de equivalencia <i>Scenario Testing</i> (reescribir los escenarios de los casos de uso como casos de prueba)	Clases de equivalencia + Análisis de valores límite Prueba de caminos (complejidad ciclomática)
Rendimiento		Simulación estadística de datos en entorno real	
Portabilidad	Lista de comprobación		Evaluar combinaciones de entorno de pruebas (hardware, S.O., base de datos, navegador, ...)
Facilidad de uso	Métodos de evaluación de usabilidad sin usuarios (comprobación de heurísticas, recorridos cognitivos)		Métodos de evaluación de usabilidad con usuarios (cuestionarios, experimentos con test de usabilidad)

Tabla de técnicas de diseño en el plan de pruebas

Característica/ Componente	Clase de riesgo	Intensidad	Tipo de pruebas	Unidades/Técnicas
Funcionalidad/ subsistema 1	B	**	Funcional	tu1: clases de equivalencia tu2: escenario testing
Funcionalidad/ subsistema 2	A	***	Funcional	tu3: prueba de caminos
Funcionalidad	B	**	Regresión	tu1 + tu3
Facilidad de uso	C	*	Usabilidad	Comprobación de heurística
Rendimiento	B	**	Rendimiento	Simulación estadística de datos en entorno real
Seguridad	A	***	Seguridad	Buscar agujeros de seguridad con técnicas de hacking

4.2. Control

- No hay cambios relevantes respecto a la fase de control dentro de la gestión global del proceso de pruebas (ver sección 3.2)



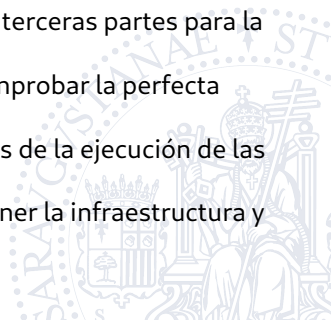
4.3. Instalación y mantenimiento de la infraestructura

- Propósito

- Proporcionar la infraestructura de pruebas que se utiliza en el resto de fases de actividades: entorno, herramientas, lugar de trabajo

- Método

- Tomando como punto de partida la infraestructura definida en el plan de pruebas, se proporcionan más detalles si es necesario
- A veces es necesario que se contacte con terceras partes para la instalación de la infraestructura
- Se crean listas de comprobación para comprobar la perfecta implantación
- La infraestructura debería estar lista antes de la ejecución de las pruebas
- También hay que preocuparse por mantener la infraestructura y preservarla para futuras pruebas



- Productos

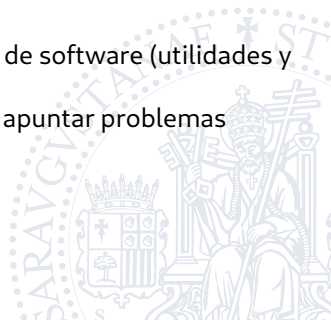
- Informe de implantación (**ver plantilla *Intake report* en TMAP**)

- Técnicas

- Listas de comprobación elaboradas en base a la experiencia de una organización para la implantación del entorno, herramientas, lugar de trabajo

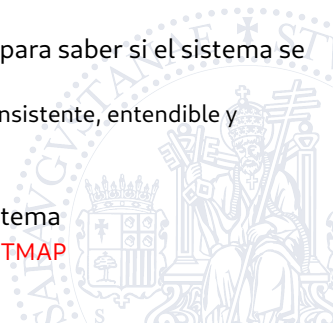
- Herramientas

- Herramienta de gestión de configuración de software (utilidades y software utilizado en las pruebas)
 - Herramienta de gestión de defectos para apuntar problemas relacionados con la infraestructura



4.4. Preparación

- Propósito
 - Acordar con el cliente una base de pruebas de suficiente calidad para diseñar los casos de prueba
- Método
 - Recopilar la base de las pruebas
 - cualquier tipo de información que define el comportamiento requerido del sistema
 - Hacer una revisión de la base de pruebas para saber si el sistema se puede probar
 - comprobar que la base de pruebas es consistente, entendible y completa
- Productos
 - Informe de la capacidad de prueba del sistema
 - *ver plantilla Testability review report en TMAP*



- Técnicas

- Listas de comprobación de información necesaria por técnica de diseño
 - *ver test design techniques en TMAP*
- Técnicas de evaluación de documentación

- Herramientas

- Herramienta de gestión de defectos para apuntar problemas relacionados con la preparación



4.5. Especificación de casos de prueba

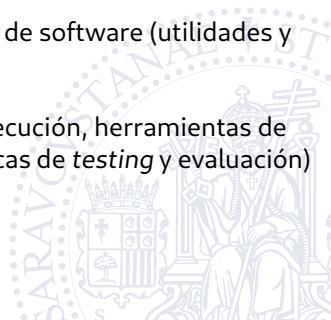
- Propósito
 - Crear los casos de prueba a partir de la base de las pruebas
 - El objetivo es tenerlo todo preparado al máximo para que la ejecución de las pruebas sea lo más rápida posible cuando los desarrolladores nos pasen el objeto de pruebas
- Método
 - Tomando como punto de partida las unidades de prueba y las técnicas de prueba asignadas y la base de pruebas, se diseñan los casos de prueba
 - También se pueden definir scripts de pruebas
 - Scripts = programas automáticos o procedimientos manuales que facilitan la ejecución de uno o varios casos de prueba de forma simple y eficiente
- Productos
 - Especificación de casos de prueba (situación de inicio, acciones, resultado esperado) y/o scripts
 - **Ver plantilla *Test script* en TMAP**

● Técnicas

- Técnicas de *testing* y evaluación (ver temas posteriores de la asignatura)
- *Ver checklists for various quality characteristics en TMAP*
(propuesta de elementos a verificar en la evaluación estática de distintas características de calidad)

● Herramientas

- Herramienta de gestión de configuración de software (utilidades y software de pruebas)
- Herramienta de gestión de defectos
- Herramientas de automatización de la ejecución, herramientas de diseño de pruebas, ... (ver temas de técnicas de *testing* y evaluación)



4.6. Ejecución

- Propósito

- Obtener detalle de la calidad del objeto de pruebas a través de la ejecución de las pruebas acordadas

- Método

- La ejecución comienza con la entrega del software a probar y su instalación en el entorno de pruebas
- Se realiza un test preliminar (pre-test) para comprobar si el sistema a probar (junto con la infraestructura) tiene suficiente calidad para continuar con la ejecución
- Si se continúa, se prepara la situación inicial de los casos de prueba, se ejecutan los scripts, y se registran los resultados
- Si hay diferencias en los resultados respecto al comportamiento esperado, se investiga
 - Si la causa es un problema en la base de las pruebas, el entorno o los casos de pruebas, se retrocede a las fases de preparación y especificación
 - Si realmente es un defecto del software probado, se registra el defecto (ver más adelante **procedimiento de gestión de defectos**)

● Productos

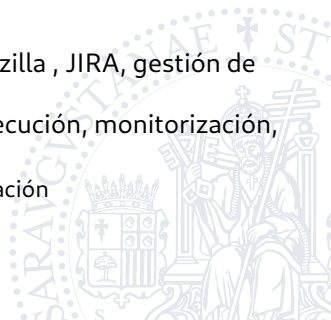
- Informes con los resultados de la ejecución de los scripts
 - hojas de control de pruebas (**ver plantilla *Test scenario* en TMAP**)
- Registro de defectos

● Técnicas

- Técnicas de *testing* y evaluación
 - ver temas posteriores de la asignatura

● Herramientas

- Herramienta de gestión de defectos: Bugzilla , JIRA, gestión de Issues en Github , ...
- Herramientas de automatización de la ejecución, monitorización, simulación, ...
 - ver temas de técnicas de *testing* y evaluación

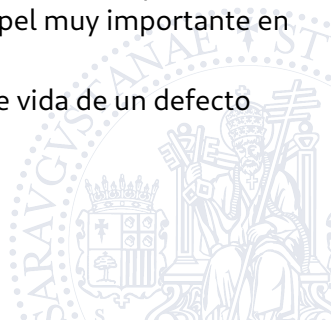


Hoja de control de pruebas

#	Nombre script	Nivel de pruebas	Componente	Objetivo de pruebas	Fecha de ejecución	Duración (si aplicable)	% ejecución	Nº defectos
1								
2								
3								
4								
5								

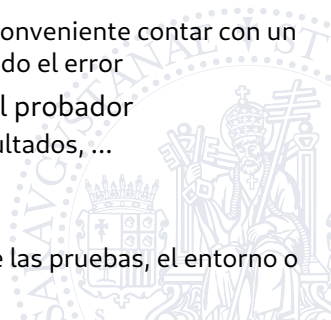


- Un defecto es la causa algorítmica de un error que conduce a un fallo del sistema (desviación del comportamiento esperado sobre el observado)
- Aunque la gestión/monitorización de defectos forma parte de la gestión del proyecto, juega también un papel muy importante en el proceso de pruebas
- Se suelen distinguir dos fases en el ciclo de vida de un defecto
 - Descubrimiento y notificación
 - Tratamiento del defecto

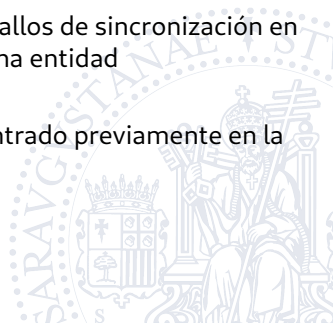


Gestión de defectos – Descubrimiento y notificación

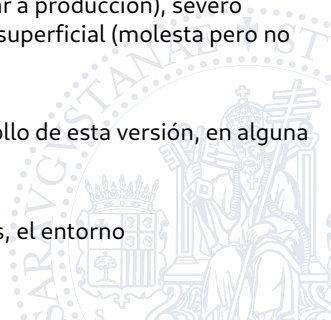
- Recopilar pruebas ante la observación de un fallo
 - Cualquier ambigüedad, inconsistencia u omisión se debe registrar mediante capturas de pantalla, copia de la base de datos, salida textual del programa, o notas del probador
- Reproducir el defecto
 - Los desarrolladores no suelen prestar mucha atención a los defectos no reproducibles
 - Pero aunque no se pueda reproducir, es conveniente contar con un histórico de las ocasiones en que ha surgido el error
- Comprobar que no sea un error interno del probador
 - Error al ejecutar la prueba, comparar resultados, ...
- Determinar la causa externa
 - El objeto a probar
 - Pero también se puede deber a la base de las pruebas, el entorno o las herramientas de prueba



- Aislar la causa
 - Si es posible, hacer variaciones sobre el caso de pruebas hasta concretar más la posible causa (errores en un parámetro/campo concreto, ...)
- Generalizar el defecto
 - Buscar otras funciones/pantallas/formularios donde se puede dar este mismo defecto
 - Ej: el mecanismo multi-usuario provoca fallos de sincronización en cualquier edición simultánea de una misma entidad
- Comparar con otros defectos
 - Comprobar si este defecto ya se ha encontrado previamente en la ejecución de otros casos de prueba



- Redactar el informe del defecto, elementos que no pueden faltar en el informe:
 - Identificación única
 - Referencia al proyecto, objeto probado, y caso de pruebas
 - Descripción, y anexos con pruebas
 - Persona que ha detectado el defecto
 - Fase o nivel de pruebas
 - Severidad (se pueden encontrar distintas clasificaciones)
 - bloqueante (el sistema no se puede llevar a producción), severo (implica costes), normal (pocos costes), superficial (molesta pero no afecta a la operatividad del sistema)
 - Prioridad
 - corrección inmediata, durante el desarrollo de esta versión, en alguna versión posterior, ...
 - Causa
 - el objeto a probar, la base de las pruebas, el entorno



- Revisar el defecto

- El gestor de pruebas, coordinador o administrador de los defectos revisa el informe para asegurar su completitud, precisión y tono
- Si todo es correcto, se somete para tratarlo y abordarlo



Gestión de defectos – Tratamiento

- Se suele crear un comité de gestión de cambios o consulta de defectos para tomar decisiones sobre los defectos registrados
- Posibles decisiones sobre los defectos
 - se abordan
 - se asignan recursos para corregir el defecto, y se vuelven a realizar las pruebas
 - se rechazan
 - Ej: los costes son demasiado altos para esta versión del sistema y solo se informa de los mismos en las notas de la versión, ...
 - se posponen
 - Ej: en el desarrollo de otra parte del sistema se utilizarán unas nuevas librerías que es probable que corrijan este defecto
 - Ej: para la siguiente versión en el entorno de producción se actualizará probablemente la versión del S.O., y no aparecerá este fallo
- Se debe actualizar el informe del defecto con los siguientes elementos:
 - Estado (nuevo, en proceso, rechazado, pospuesto, resuelto, ...)
 - Persona que resuelve el defecto
 - Solución adoptada

4.7. Finalización

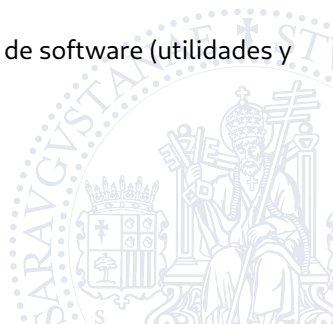
- Propósito
 - Aprender de la experiencia de este nivel de pruebas, y garantizar la reutilización de las pruebas en el futuro
- Método
 - Se evalúa cómo ha ido el proceso de pruebas (los resultados de la evaluación sirven como entrada para los informes de la fase de control)
 - Se preserva la infraestructura de las pruebas para futuras pruebas de mantenimiento, regresión, ...
- Productos
 - Contribución a los informes de progreso e informes finales de la fase de control
 - Software de las pruebas reutilizable controlado

- Técnicas

- Lista de comprobación de problemas típicos que pueden surgir durante el proceso de pruebas
 - *Ver process evaluation en TMAP*

- Herramientas

- Herramienta de gestión de defectos
- Herramienta de gestión de configuración de software (utilidades y software de pruebas)



Índice

1. Conceptos básicos
2. Fundamentos de TMAP
3. Gestión global del proceso de pruebas
4. Pruebas de sistema y aceptación
5. Pruebas de desarrollo



5. Pruebas de desarrollo

- No se pueden organizar como un proceso autónomo para un equipo independiente, sino que se deben coordinar con el desarrollo y el equipo de desarrollo
- Motivos:
 - Implican la utilización del conocimiento técnico de la implementación del sistema
 - Son una parte integral del trabajo de un desarrollador
 - Es habitual que la persona que detecta los errores sea la misma que los resuelve
- Pero sí se pueden identificar y describir diferentes actividades con ayuda del ciclo de vida de TMAP
- Es importante el concepto de calidad acordada con el cliente
 - Acordar prácticas de desarrollo orientadas a la mejora de la calidad

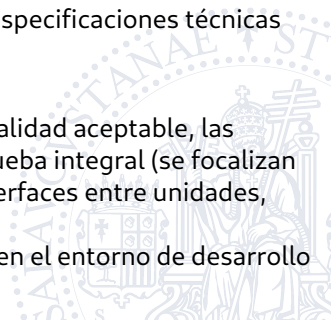
Distinción habitual entre pruebas de unidad y de integración

- Pruebas de unidad

- Las pruebas de desarrollo empiezan con la prueba de las partes más pequeñas del sistema (rutinas, unidades, programas, módulos, objetos)
- Las realizan los desarrolladores en el entorno de desarrollo
- Se verifica que la unidad cumple con las especificaciones técnicas (diseño detallado)

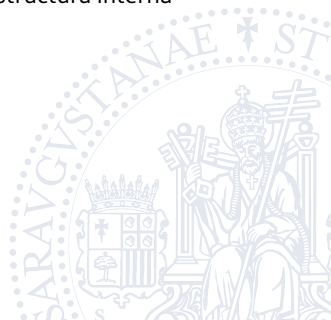
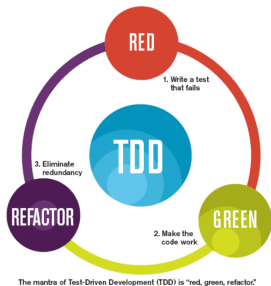
- Pruebas de integración

- Cuando las partes pequeñas tienen una calidad aceptable, las partes más grandes se someten a una prueba integral (se focalizan en la producción de datos y en probar interfaces entre unidades, subsistemas, ...)
- Las realizan también los desarrolladores en el entorno de desarrollo

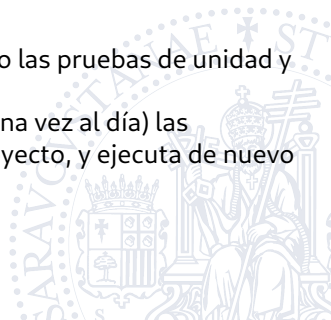


Prácticas de desarrollo orientadas a la mejora de la calidad

- Desarrollo dirigido por las pruebas (*Test-Driven Development*)
 - Método planteado en la metodología (ciclo de vida) de Programación Extrema (*eXtreme Programming*)
 - Iterativamente se realiza lo siguiente
 - Escritura de pruebas (que inicialmente fallan)
 - Construir código mínimo para pasar las pruebas
 - Refactorizar el código para mejorar su estructura interna



- Programación en pareja (*Pair programming*)
 - Método de desarrollo en el que dos desarrolladores trabajan en el mismo código y también especifican y ejecutan conjuntamente las pruebas de unidad
- Revisión del código
 - Inspeccionar y revisar el código frente a las especificaciones, y sometiéndolo también a revisión por pares, para mejorar su calidad
- Integración continua
 - Aproximación donde se han automatizado las pruebas de unidad y las de integración
 - El desarrollador integra con frecuencia (una vez al día) las modificaciones que ha realizado en el proyecto, y ejecuta de nuevo todas las pruebas (unidad e integración)



¿Cuestiones?



Departamento de
Informática e Ingeniería
de Sistemas
Universidad Zaragoza

